C-Factor Estimation from Sentinel-2 Fractional Cover

Reference Documentation — Calibration and ML Pipeline

Sélène Ledain

June 2026

Contents

1 Introduction and Background

The C-factor (cover-management factor) is one of the six factors in the Revised Universal Soil Loss Equation (RUSLE; **?**):

$$A = R \cdot K \cdot L \cdot S \cdot P \cdot C$$

where A is the annual average soil loss, R the rainfall and surface runoff factor, K the soil erodibility factor, L the slope length factor, S the slope gradient factor, P the erosion control factor and C the soil cover and cultivation factor. C captures the combined effect of vegetation cover, crop rotation, and tillage on soil detachment.

In the existing Swiss erosion risk pipeline (Prasuhn et al., 2023), C is assigned from a tabulated lookup that maps each crop type to a single value that depends on region (Tal/Berg, i.e. lowland/upland) and tillage practice (Pflug/Mulch/Direkt, i.e. plowing, mulching, direct seeding). These tabulated values were established by determining soil loss ratios (based on literature and some measurements) for crops at different growth stages, combining this to rainfall erosivity data, accounting for certain crop rotations and interactions, as well as specific soil preparation practices and altitude. For more details on how these C-factors were determined, please refer to Prasuhn et al. (2023). These C-factors do not account for changing environmental conditions, per-field spatial and year-to-year temporal variability, therefore representing in a limited way what actually occurs on the field.

The C-factor is computed by convolving a time series of the *soil loss ratio* (SLR) with the daily rainfall erosivity index (EI):

$$C = \frac{\sum_t \mathrm{SLR}(t) \cdot \mathrm{EI}(t)}{\sum_t \mathrm{EI}(t)}. \tag{1}$$

SLR itself is modelled as

$$\mathrm{SLR}(t) = \exp\big(-\beta \cdot \mathrm{FC}(t)\big), \tag{2}$$

where $\mathrm{FC}(t)$ is fractional vegetation cover (both photosynthetic vegetation and crop residues, scaled to 0–100 %) and β is an empirical decay parameter (Matthews et al., 2023).

The motivation for this project is to replace the static tabulated C-values with a satellite-driven product that:

- captures within-crop spatial and inter-annual variation in cover dynamics that the lookup table cannot encode;
- remains anchored to the tabulated reference values so the operational product is consistent with the previous knowledge on erosion;
- can be updated based on new satellite and climatology data.

We developed two approaches to predict the C-factor:

1. **Empirical calibration**: calibrates the C-factor equation using satellite-derived FC and updated EI data to match the tabulated C-factor values. A scalar parameter(s) β in the Soil Loss Ratio (SLR) model $\mathrm{SLR}(t) = \exp(-\beta \cdot \mathrm{FC}(t))$ is calibrated against the tabulated per-crop reference values, following Matthews et al. (2023). (Section 5)
2. **Machine-learning prediction**: a multilayer perceptron (MLP) maps the full $(\mathrm{PV}(t), \mathrm{NPV}(t), \mathrm{EI}(t))_t$ time series directly to the tabulated C_{ref}. The model learns the relationship directly from time series of FC and EI, with the tabulated C-factors used as labels during training. (Section 6)

Both approaches use the same underlying data but differ in how the relationship between predictors and C-factor is established. The results of the calibration/training are presented in Section 7. Once the models are established, they are used to predict C-factors over whole regions

and across years at a Sentinel-2 pixel level (Section 9). These results can then be aggregated to farm or municipality level. An analysis for the district of Seeland in canton Bern is presented in Section **??**, where the new products are assessed in comparison to the previously obtained C-factors.

2 Repository Structure

The relevant code lives in the `/mnt/eo-nas1/eoa-share/projects/028_Erosion/Erosion/cfactor/` sub-directory.

File	Purpose
`main.py`	Entry point; runs sampling + calibration.
`sample_FC.py`	Sampling, FC extraction, cleaning, GPR gap-fill.
`calibrate_cfactor.py`	EI joining, β calibration, stratified calibration.
`check_sampled.py`	Diagnostics for the sampled dataset.
`analyse_calibration_sample.py`	Post-hoc analysis of empirical calibration.
`tune_cfactor_ml.py`	Hyper-parameter search for ML. models.
`train_cfactor_ml.py`	ML model training
`analyse_ml_sample.py`	Post-hoc analysis of ML model.
`weighted_mlp.py`	Custom `WeightedMLPRegressor` wrapper.
`compare_beta_vs_ml.py`	Empirical vs ML comparison.
`compute_cfactor_pixels.py`	Per-pixel annual C-factor inference using the calibrated β (operational).
`compute_cfactor_pixels_ml.py`	Per-pixel annual C-factor inference using the trained ML pipeline (operational).
`agis_field_filtering.py`	Standalone utility to explore the AGIS farm-level selection rules used in `sample_FC.py`.
`group_crops.py`	Utility to inspect crop groupings used in `sample_FC.py`.
`c_factor_provenance.xlsx`	Provenance table (which C-values are estimated).

3 Data Requirements

The datasets used throughout the project are listed here below (paths relative to the `EO-NAS`).

3.1 Inputs

LNF polygon maps

`/mnt/eo-nas1/data/landuse/raw`

Annual land-use maps (`lnf{year}.gpkg`), one per year, containing field polygons with `lnf_code` attribute (agricultural crop type code). Years 2019–2025 are supported; the LNF field-id column is `uuid` for $\leq$2022 and `identifikator_be` for $\geq$2023.

AGIS Nutzungsdaten

`/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected/Daten/Core_Snapshot/Agrarbericht_2025/tbl_nutzungsdaten.csv`

AGIS field-parcel table with columns `Flaechen_ID`(field id), `betr_ID` (farm id), `Jahr`, `Kultur_nutzung`, `flaeche_bewirt`, `swissALTI3D`. Used for field sampling and altitude-based stratum assignment.

AGIS REB table

`/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected/`

Daten/Core_Snapshot/Agrarbericht_2025/tbl_ressourceneffizienzbeitrag.csv
Payments table with columns betr_ID, Jahr, reb_sb (tillage type: Mulchsaat, Direktsaat, Streifensaat, or NA for Pflug).

LNF classification spreadsheet
/mnt/eo-nas1/data/landuse/documentation/
LNF_code_classification_20260217.xlsx
Excel workbook with sheet label_sheet containing LNF_code, Crop_DE, Crop_EN, Crop_Label_lv3 (distinguishing arable crops, grasslands and other), and per-year area columns (YYYY_Area_mYY in m^2).

LNF culture mapping CSV
/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected
/Daten/Core_Snapshot/Agrarbericht_2025/tbl_kulturmapping.csv
kulturcode ↔ Kultur_nutzung (used to join AGIS crop names to LNF codes).

Sentinel-2 raw data
/mnt/eo-nas1/data/satellite/sentinel2/raw/CH
Zarr archives per tile per year,S2_{left}_{top}_{startdate}_{enddate}*.zarr, with bands s2_B02–s2_B12, s2_SCL, s2_mask.

Pre-computed FC (optional)
/mnt/eo-nas1/data/satellite/sentinel2/FC
Zarr archives with variables PV, NPV, Soil in the same tile naming convention. If missing, FC is predicted on-the-fly from S2 bands using the trained ensemble models.

FC models ~/mnt/eo-nas1/eoa-share/projects/012_EO_dataInfrastructure/SALI_models
Soil-adjusted ensemble of five PyTorch models per soil group. Please refer to report on Fractional Cover models for more on this.

DLR soil group map
/mnt/eo-nas1/data/satellite/sentinel2/DLR_soilsuite_preds
Per-tile Zarr SRC_{left}_{top}.zarr with variable soil_group, used to select the correct FC model.

EI grid

C-factor reference table
/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected
/Daten/Erosionsrisiko/C_Faktoren.csv
Tabulated per-crop C-factors (Prasuhn et al., 2023) with columns Kultur Kategorien 2020, Total, Tal_Pflug, Tal_Mulch, Tal_Direkt, Berg_Pflug, Berg_Mulch, Berg_Direkt.

BLW CSV /mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected/
Daten/Erosionsrisiko/schonende_bodenbearbeitung.csv
BLW list of fields enrolled in conservation-tillage programmes (Mulchsaat, Direktsaat, Streifensaat). Used to additionally sample fields with known management (Section 4.4). Columns include betr_ID (matching the AGIS farm identifier), Flaechen_ID, Jahr, and a tillage-type column.

S2 tile grid /mnt/eo-nas1/eoa-share/projects/012_EO_dataInfrastructure/Project layers/gridface_s2tiles_CH.shp
Shapefile with geographic extent of S2 zarr tiles and identifiers (left, top).

3.2 Intermediate outputs

File	Content
`samplesv2.pkl`	GeoDataFrame: one sampled point per field-year (AGIS + BLW).
`samples_datav2.pkl`	S2 pixel values for all sampled fields.
`samples_data_predv2.pkl`	FC predictions (PV, NPV, SOIL) per pixel-date.
`samples_data_gprv2.parquet`	GPR gap-filled FC on regular 10-day grid.

Note: file names are set by the `samples_path`, `samples_s2_path`, `fc_preds_path`, and `gapfilled_fc_path` keys in `CONFIG` in `main.py`; the `v2` suffix distinguishes the current dataset from earlier runs.

4 Part I: Sampling and Gap-Filling Pipeline

Run with:

```
python main.py #comment out run_calibration() to do only the sampling
    part
```

The sampling pipeline is implemented in `sample_FC.py` and called from `main.py`. Its purpose is to build a representative training dataset of Sentinel-2 fractional cover time series, that will be used to calibrate/train the C-factor models. The `CONFIG` keys referred to are set in `main.py`.

4.1 Step 1 – Top crop indentification

The pipeline begins by identifying the top-20 arable crops by national agricultural area. Only crops labelled `Arable Land` in the LNF classification spreadsheet (`Crop_Label_lv3`) are considered. Area is read from the per-year columns of the classification spreadsheet (columns named `YYYY_Area_mYY`. Only years from 2022 onwards are used since the LNF polygons for years prior to that were incomplete, resulting in false computation of the area. A union of the top-20 crops across all requested area years is taken, so the selection is robust to year-to-year fluctuations.

Certain LNF codes are explicitly excluded (`CONFIG` `lnf_ignore_codes`, e.g. orchard or special-use codes). Codes listed in `grassland_codes` are removed from the arable list even if the lv3 label says "Arable Land" (e.g. code 601, *Kunstwiesen*, which is labelled as arable but agronomically behaves as grassland and can be sampled as a grassland baseline separately).

4.2 Step 2 – Crop grouping by identical C-factors

Crops whose entire stratified C-factor vector ($C_{\text{Tal,Pfl}}$, $C_{\text{Tal,Mul}}$, $C_{\text{Tal,Dir}}$, $C_{\text{Berg,Pfl}}$, $C_{\text{Berg,Mul}}$, $C_{\text{Berg,Dir}}$) is identical to a top crop are pooled into that crop (*analogies*). This widens the field universe for less common crops without inflating the calibration target (the reference C is the same).

The grouping logic (`build_crop_groups` in `sample_FC.py`):

1. Loads the C-factor table and the LNF–crop bridge via `tbl_kulturmapping.csv`.
2. Drops codes flagged as fully estimated (`n_estimate == 6`) in `c_factor_provenance.xlsx`.
3. Groups LNF codes sharing an identical 6-element C-factor vector.
4. Finds connected components in the resulting graph and maps each analogy code to the lowest-numbered top crop in its component.

Analogy polygons carry their true LNF code in `orig_lnf_code` and the main code in `lnf_code` throughout the rest of the pipeline.

4.3 Step 3 – Sampling strategy

Fields are sampled using the AGIS-informed strategy (`sampling_strategy: 'agis'` in `CONFIG`). The AGIS and BLW datasets are used to identify fields with known (or assumed with high certainty) soil preparation (plow, mulch, direct seeding). A simpler area-weighted random draw directly from the LNF polygons is also available (`'random'`), but it may include fields where the declared crop code does not reliably reflect what was planted (mixed farms, intercropping) and cannot be used to calibrate the detailed C-factor (`'stratified_calibration'`, per altitude and soil preparation method). The AGIS strategy is preferred for calibration because it reduces label noise; see Section 11 for the rationale.

The AGIS strategy uses `tbl_nutzungsdaten` to build a shortlist of "clean" arable fields satisfying one or more of the following farm-level rules (applied as a union, controlled by `agis_rules` in `CONFIG`):

1. `single_crop_farm`: the farm grows exactly one crop, which must be a top arable crop.
2. `grassland_plus_one`: the farm is predominantly grassland ($\geq$ `agis_grass_min_share`, default 50%) with exactly one top arable crop covering $\geq$ `agis_arable_min_share` (default 2%) and all other land types $\leq$ `agis_other_max_share` (default 20%).
3. `dominant_crop`: a single top arable crop covers $\geq$ 80% of farm area (threshold set by `agis_dominant_share`).

Analogy crops are folded into their main crop before the farm-level rules are applied, so a farm growing code 531 and its C-identical analogy 592 is treated as a single-crop farm.

The shortlist is joined onto the LNF polygon layer via `Flaechen_ID` (the field identifier shared between AGIS and LNF). Grassland codes are sampled separately with the standard random strategy (AGIS rules target arable crops only). One point per polygon is drawn by buffering the polygon inward by 10 m (to avoid edge effects) and sampling uniformly within the buffered geometry. The resulting sampled point carries: `poly_id` (polygon index), `lnf_code`, `orig_lnf_code`, `x`, `y`, `yr`, `uuid` (AGIS `Flaechen_ID`), and `betr_ID` (farm identifier). The last two are required for stratified calibration (Section 5.4).

4.4 Step 3b – BLW augmentation

Whenever `sampling_strategy == 'agis'`, a second sampler runs automatically and adds fields with *known* conservation-tillage class from the BLW dataset (`verfahren_path`). Because the REB table records tillage at the farm–year level (not the field level), there is ambiguity in the main AGIS sample about which tillage method was applied to a specific field. This dataset esolves this ambiguity for enrolled fields: Mulchsaat, Direktsaat, and Streifensaat are directly recorded per field.

Up to `verfahren_tot_samples` (default 5000) field-years are drawn from the file, stratified by `verfahren_stratify_cols` (default: `lnf_code`, `tillage_class`) so that the draw is balanced across crops and tillage types. Only years listed in `verfahren_years` (default: 2021, 2022) are used.

This new sample is then merged with the AGIS sample by deduplication on `(uuid, yr)`: when a field-year appears in both sources, the BLW row wins so the directly known tillage class is preserved. The resulting `tillage_class` column (values: Mulch, Direkt, Pflug) flows through to the gapfilled parquet. Note that `calibrate_cfactor.py`'s `assign_strata` function currently assigns tillage from the REB table for all pixels regardless of whether a `tillage_class` value is already known; the column is preserved in the parquet for future use.

The same crop-grouping (analogy folding) as applied to the AGIS sample is also applied to the BLW draw.

4.5 Step 4 – Sentinel-2 extraction and FC prediction

For each sampled field-year, the pipeline clips the S2 data to the polygon and extracts all pixels over the agricultural year window (July 1 of year−1 to June 30 of year, following Prasuhn et al. (2023)). The S2 data includes all spectral bands (`s2_B02`–`s2_B12`), the scene classification layer (`s2_SCL`), and a cloud mask (`s2_mask`).

On-the-fly FC prediction If pre-computed FC is not used, the raw S2 bands are extracted and fractional cover is predicted using the FC ensemble model. The prediction is performed per soil group (identified through the precomputed soil group predictions passed in 'soil_group') and the output is renormalised so that PV + NPV + Soil = 1.

> **Note:** There is a config key `fc_precompute` which tells the code whether to use precomputed and saved FC values, or whether to predict them on the fly. The precomputed route is not fully coded, because it is less optimal: each FC zarr file still needs to be opened with its original Sentinel-2 zarr file in order to extract cloud masks for downstream data cleaning. the on the fly prediction actually runs quite fast and is advised.

4.6 Step 5 – Cloud/shadow/snow cleaning

Each pixel-date is flagged for removal if any of the following apply (`clean_timeseries_field`):

- All S2 bands are NaN or saturated (65535).
- `s2_mask` = 1 (cloud) or `s2_SCL` $\in \{8, 9, 10\}$ (cloud medium/high probability, cirrus).
- `s2_mask` = 2 (shadow) or `s2_SCL` = 3.
- `s2_mask` = 3 (snow) or `s2_SCL` = 11.
- `s2_SCL` = 10 and `s2_B02` > `cirrus_thresh` (default 500) – additional cirrus detection via the blue band.

A date is dropped from the time series if more than `max_missing_frac` (default 5%) of the field's pixels are masked. An entire field-year is dropped if more than `drop_fraction_threshold` (default 70%) of all dates are removed, as there would be insufficient clean observations for reliable gap-filling. These thresholds were set through trial and error.

4.7 Step 6 – GPR gap-filling

After cleaning, the per-pixel time series is gap-filled using Gaussian Process Regression (GPR). The GPR predicts FC on a regular grid (`'regular'` mode in `CONFIG`) of `grid_step_days` (default 10 days) anchored at July 1 of year−1. This produces a uniform 37-step time series per field-year, which is required for the ML feature matrix (Section 6.2).

If `'irregular'` mode is set, the cleaned observations are kept as-is. GPR fills only gaps longer than `max_gap_days` (default 15 days) by inserting synthetic dates at regular intervals within the gap. This preserves the original uneven observation cadence. However, this does not work with the ML pipeline and doesn't ensure a same number of observation per pixels.

In both modes the GPR is fitted per field-year using the following design:

- **Training data**: all clean pixels of the field (not just the target pixel), up to 1000 points. When more are available, a stratified subsample is drawn (always keeping all observations of the target pixel, remaining budget drawn from a temporal bins of other pixels to preserve coverage).
- **Response**: additive log-ratio (ALR) transform of $(\mathrm{PV}, \mathrm{NPV}, \mathrm{Soil})$, using Soil as the reference component. The GPR models the two ALR components independently; the soil component is recovered as $1 - (\mathrm{PV} + \mathrm{NPV})$ after back-transformation.
- **Features**: normalised time t (days since grid start / 365), seasonality $(\sin(2\pi t/365), \cos(2\pi t/365))$, and standardised pixel coordinates (x, y).
- **Kernel**: $C(1, \cdot) \times \mathrm{Matérn}_{3/2}(l\_t, l\_\sin, l\_\cos, l\_x, l\_y) + \mathrm{WhiteKernel}(\sigma_n)$, with length-scale bounds tuned per component (tighter for temporal features, looser for spatial). The noise level is estimated from the variance of the ALR training data.
- **Optimisation**: `n_restarts_optimizer=5` for the first ALR component; the fitted kernel is re-used for the second component with 2 restarts.

Each gap-filled or grid point receives a `fc_quality_flag`: 0 = well-interpolated, 1 = higher uncertainty (>80th percentile of ALR uncertainty), 2 = very high uncertainty (>95th percentile).

The final output (`gapfilled_fc_path` in `CONFIG`, currently `samples_data_gprv2.parquet`) contains one row per target-pixel grid point: `lnf_code, yr, poly_id, x, y, time, pv, npv, soil, is_gapfilled, fc_quality_flag`, plus `fc_total` = (PV + NPV) × 100, and (AGIS mode) `uuid`, `betr_ID`, and `tillage_class` (from the BLW augmentation where available).

4.8 Checking sampled data

After sampling, `check_sampled.py` produces a set of diagnostic plots in `sample_analysisv2/`. It is organised into three sections:

Section A Sampling overview: map of sampled pixel locations and stacked per-crop sample counts by year.

Section B Cleaning diagnostics: per-row mask attribution by month, drop-fraction histogram, pre/post-cleaning PV seasonal cycles, pipeline funnel (field counts and temporal coverage at each stage), per-crop median coverage, and a sensitivity sweep over `max_missing_frac` and `drop_fraction_threshold`.

Section B' Post-cleaning sampling overview: mirrors Section A but restricted to field-years that survived all filters. Includes a crop-grouping view showing the composition of each analogy-pooled group (which `orig_lnf_code` values were folded into each main code) and side-by-side raw-vs-cleaned retention rates per crop.

Section C Gapfilling diagnostics: fill-point accounting, gap-length distributions, distribution validity checks (monthly observed vs. gapfilled means per FC component), composition validity (PV + NPV + Soil = 1), quality-flag breakdown, and a held-out cross-validation that drops one clean observation per field, re-fits the GPR, and checks the prediction accuracy (`gpr_cv_scatter.png`).

Run with:

```
python check_sampled.py
```

5 Part II: Empirical Calibration

Run with:

```
python main.py --skip-sampling
```

5.1 Model formulation

Following Matthews et al. (2023), the per-pixel annual C-factor is estimated as the EI-weighted mean of the Soil Loss Ratio:

$$C_{\text{pixel}} = \frac{\sum_t \text{SLR}(t) \cdot \text{EI}(t)}{\sum_t \text{EI}(t)} \tag{3}$$

where the sum is over all time steps of the agricultural year (July 1 of year-1 to June 30 of year). The SLR is modelled as:

$$\text{SLR}(t) = \exp\big(-\beta \cdot \text{FC}(t)\big), \quad \text{FC}(t) = (\text{PV}(t) + \text{NPV}(t)) \times 100. \tag{4}$$

Single-β mode A single scalar β multiplies total fractional cover. This matches the Matthews et al. formulation; their reported value is $\beta \approx 0.04$ (with FC on the 0–100 scale). Selected with `calibration_mode: 'single'`.

Two-β mode (default) Separate coefficients $(\beta_{\text{PV}}, \beta_{\text{NPV}})$ are fitted for green and non-photosynthetic vegetation:

$$\text{SLR}(t) = \exp\big(-\beta_{\text{PV}} \cdot \text{PV}(t) \cdot 100 - \beta_{\text{NPV}} \cdot \text{NPV}(t) \cdot 100\big).$$

The two-β model reduces to the single-β model when $\beta_{\text{PV}} = \beta_{\text{NPV}}$. The two-$\beta$ mode is motivated by the hypothesis that NPV (residues, senescent material) provides erosion protection differently from green cover. Selected with `calibration_mode: 'two_beta'`. Per-component search bounds can be set independently with `beta_bounds_pv` and `beta_bounds_npv` (each falls back to `beta_bounds` if `None`).

5.2 Rainfall erosivity index (EI)

Equation (3) uses climatological average daily EI values from a 100 m gridded parquet file. For each sampled FC pixel, the (x, y) coordinate is snapped to the nearest EI grid cell. The same long-term-average EI is reused across all years, which is appropriate when comparing against a reference table built against climatological EI. For more on the generation of this data, refer to https://github.com/EOA-team/erosion_indicator.

5.3 Reference C-factor loading

The reference C-factors are read from `C_Faktoren.csv`. LNF codes are bridged to C-factor table entries via the German crop name (`Crop_DE` from the LNF classification spreadsheet ↔ `Kultur Kategorien 2020` in `C_Faktoren.csv`). A normalised-form fallback handles minor punctuation differences (commas, parentheses, etc.). Any remaining mismatches can be resolved with a `manual_overrides.csv` file (columns: `lnf_code,crop_name`).

Area weights When `area_weight_loss: True`, the calibration loss is weighted by Swiss arable area per crop (same as used for sampling: mean of `YYYY_Area_mYY` columns across `area_years`, converted to hectares). This ensures that cereals (which dominate Swiss arable area) drive the fit rather than minor crops that cover little area.

5.4 Calibration procedure

The calibration target is the stratified C-factor table (`stratified_calibration: True` in `CONFIG`), with six columns ($C_{\text{Tal,Pfl}}, C_{\text{Tal,Mul}}, C_{\text{Tal,Dir}}, C_{\text{Berg,Pfl}}, C_{\text{Berg,Mul}}, C_{\text{Berg,Dir}}$). Setting `stratified_calibration: False` switches to the simpler unstratified mode, where each crop has a single reference value (`Total` column) and pixel predictions are averaged to one value per crop before computing the loss. The stratified mode is preferred because it exploits the additional information in the reference table and uses the `uuid`/`betr_ID` identifiers collected during AGIS + BLW sampling.

The calibration procedure is:

1. Compute $C_{\text{pixel}}(\beta)$ via Equation (3) for all sampled pixels, carrying the stratum labels (`region`, `tillage`) assigned below.
2. Average pixel-level predictions to one value per (crop, region, tillage) stratum.
3. Compute the (area-weighted) MAE across strata: $\text{MAE}(\beta) = \sum_{c,r,t} w_{c,r,t} \, |C_{\text{pred,c,r,t}} - C_{\text{ref,c,r,t}}|$.
4. Minimise $\text{MAE}(\beta)$ w.r.t. β using `scipy.optimize.minimize_scalar` with the bounded Brent method (single-β) or `L-BFGS-B` (two-β).

Each sampled pixel is assigned a stratum based on:

Altitude (Tal/Berg) from the `swissALTI3D` column of `tbl_nutzungsdaten`, joined by (`Flaechen_ID = uuid`, `Jahr = yr`). Pixels ≤ 600 m are Tal; > 600 m are Berg. The cutoff is set by `grenze_tal_berg` (default 600 m) to match strategy used in the previous C-factor calculations.

Tillage (Pflug/Mulch/Direkt) from `reb_sb` in `tbl_ressourceneffizienzbeitrag`, joined by (`betr_ID`, `Jahr`). Raw values are mapped: Mulchsaat $\rightarrow$ Mulch, Direktsaat/Streifensaat $\rightarrow$ Direkt, NA $\rightarrow$ `standardansaatverfahren` (default: Pflug).

Because the REB table is farm-year-keyed (not field-keyed), within-farm-year mixed tillage is ambiguous. This is why the AGIS+BLW filtering was done, to maximise the chance of there being one crop/being sure which management occured. The `tillage_assignment` config knob controls the resolution:

- `'stochastic'` (default): draw per pixel from the empirical tillage frequency distribution of the farm-year. Zero bias in expectation.
- `'first_row'`: use the first row's `reb_sb` (matches the method used previously in MAUS R pipeline, in `05-Dataprep.R`).
- `'mode'`: use the most frequent `reb_sb` value.

The stratified loss uses one residual per non-empty (crop, region, tillage) stratum, with weights $w_{c,r,t} \propto A_c \cdot n_{\text{pixels}\,c,r,t} / n_{\text{pixels}\,c}$ (proportional to crop area times the stratum's share of that crop's pixels). This preserves each crop's contribution to the loss equal to its unstratified contribution.

5.5 Outputs

Results are written to the folder specified by `results_folder` in `CONFIG`.

File	Content
`beta_stratified.json`	Optimal $(\beta_{\text{PV}}, \beta_{\text{NPV}})$ for two-β mode, or scalar β for single mode.
`calibration_results_stratified.csv`	Per-(crop, region, tillage) C_{pred}, C_{ref}, residual.
`calibration_results_stratified_per_pixel.csv`	Per-pixel C_{pred}.
`calibration_scatter_stratified.png`	Predicted vs reference scatter.
`beta_sensitivity_stratified.png`	MAE vs β per crop (Matthews et al. Fig. 2 style).

5.6 Post-hoc analysis

`analyse_calibration_sample.py` reads the per-pixel calibration output above (`calibration_results_per_pixel.csv`) and produces in-sample diagnostic plots and summary tables for the parcels that were used to fit β. It auto-detects stratified mode from the presence of the `region` and `tillage` columns (override with `'stratified_mode'`); in stratified mode every per-crop plot is replaced by its per-stratum equivalent. Set `results_dir` at the top of the file to the calibration output folder and run:

```
python analyse_calibration_sample.py
```

It writes the following figures to `<results_dir>/plots/`:

- `cal_scatter_per_crop.png` — crop-mean predicted vs reference C-factor.
- `cal_strengths_weaknesses.png` — per-crop bias against within-crop spread.
- `cal_box_per_crop.png` — boxplot of per-pixel C_{new} per crop, with the reference C_{ref} marked.
- `cal_interannual.png` — C_{new} by year per crop.
- `cal_residual_vs_n.png` — |bias| against sample size per crop.
- `cal_quality_bars.png` — bias/MAE per FC quality flag (only if the FC parquet is available).

and the underlying summary table to `<results_dir>/` (`cal_per_crop.csv`, or `cal_per_stratum.csv` in stratified mode).

5.7 Best calibration variant

The calibration (Section 5) was run on the AGIS+BLW sampled dataset from Part I (`sampled_data_gprv2.parquet`: cleaned, gap-filled, pixel-level FC time series spanning 1 July of the previous year to 30 June of the current year, joined to climatological EI). Four variants were evaluated, crossing the SLR parameterisation with the treatment of the ley / artificial-meadow class (LNF codes 601, 602, excluded via `exclude_calibration_lnf_codes`):

- **Single** β — one global β, $\text{SLR} = \exp(-\beta\,\text{FC})$.
- **Two** β — separate β_{PV}, β_{NPV}.
- **Single** β**, no ley** — as above, ley excluded from the fit.
- **Two** β**, no ley** — as above, ley excluded from the fit.

Each variant is scored by the calibration objective itself: the mean absolute error (MAE) between predicted and reference C-factors aggregated to the crop / stratum level (Section 5.4), area-weighted by crop extent.

The lowest cross-crop MAE was obtained by the **two-β, no-ley** variant; its calibrated parameters and per-crop diagnostics are stored in `calibration_analysis_twobeta_noley/` (`beta_stratified.json`, `calibration_results.csv`). This is the β artefact loaded by the operational product in Section 9.

Variant	MAE	Bias	Output folder
Single β	*0.0589*	*-0.035*	`calibration_analysis_single`
Two β	*0.0592*	*-0.0372*	`calibration_analysis_twobeta`
Single β, no ley	*0.0525*	*-0.0036*	`calibration_analysis_single_noley`
Two β, no ley	**0.0509**	**0.0034**	`calibration_analysis_twobeta_noley`

Table 1: Empirical calibration variants, scored by area-weighted cross-crop MAE (lower is better). Selected variant in bold.

6 Part III: ML-Based C-Factor Prediction

6.1 Motivation

The empirical approach imposes the parametric form $\text{SLR} = \exp(-\beta \cdot \text{FC})$ and fits one scalar. An ML model instead learns the mapping $(\text{PV}(t), \text{NPV}(t), \text{EI}(t))_t \to C_{\text{ref}}$ directly from data, with no prescribed functional form. This allows the model to exploit temporal patterns (e.g. the timing of residue cover or peak greenness) that the scalar β cannot capture. The trade-off is that the ML model must be regularised to avoid overfitting the data.

6.2 Feature engineering

Each sampled pixel-year is represented by a flat feature vector formed by pivoting the regular-grid time series (`gapfill_method='regular'`, 37 time steps at 10-day cadence):

$$\mathbf{x} = [\underbrace{\text{pv}_{t_0}, \ldots, \text{pv}_{t_{36}}}_{37}, \underbrace{\text{npv}_{t_0}, \ldots, \text{npv}_{t_{36}}}_{37}, \underbrace{\text{ei}_{t_0}, \ldots, \text{ei}_{t_{36}}}_{37}] \in \mathbb{R}^{111}.$$

Column names follow the pattern `pv_t000`, `npv_t000`, `ei_t000`, ... All features are standardised (zero mean, unit variance) before model fitting.

Important: the regular-grid gap-fill (`gapfill_method= 'regular'` in the sampling part) is required for ML features because the pivot step fails if time series have unequal lengths.

6.3 Hyperparameter tuning

Run:

```
python tune_cfactor_ml.py
```

The `CONFIG` keys referred to are set in `tune_cfactor_ml.py` and `train_cfactor_ml.py`. When run directly, `tune_cfactor_ml.py` imports `CONFIG` from `train_cfactor_ml.py`. The data-assembly keys that tune reads from it — and that must be consistent between tuning and training — are: `gapfilled_fc_path`, `ei_path`, `c_factor_table_path`, `stratified_calibration`, `exclude_calibration_lnf_codes`, `area_weight_loss`, `area_years`, `split_strategy` (see bloew), and (if stratified) the tillage/altitude keys. Changing any of these in train's CONFIG automatically changes what tune optimises for.

Tune-specific behaviour can be overridden by adding two optional keys to train's CONFIG:

- `tune_results_folder` — output directory (default: `'calibration_analysis_tune'`).
- `tune_params` — nested dict overriding any of `cv_group` (`'crop'` or `'stratum'`, default `'stratum'`; ignored when `split_strategy == 'stratified'`), `n_splits`, `random_state`, `baselines`, `grid` (architecture×α search space), and `alpha_curve`.

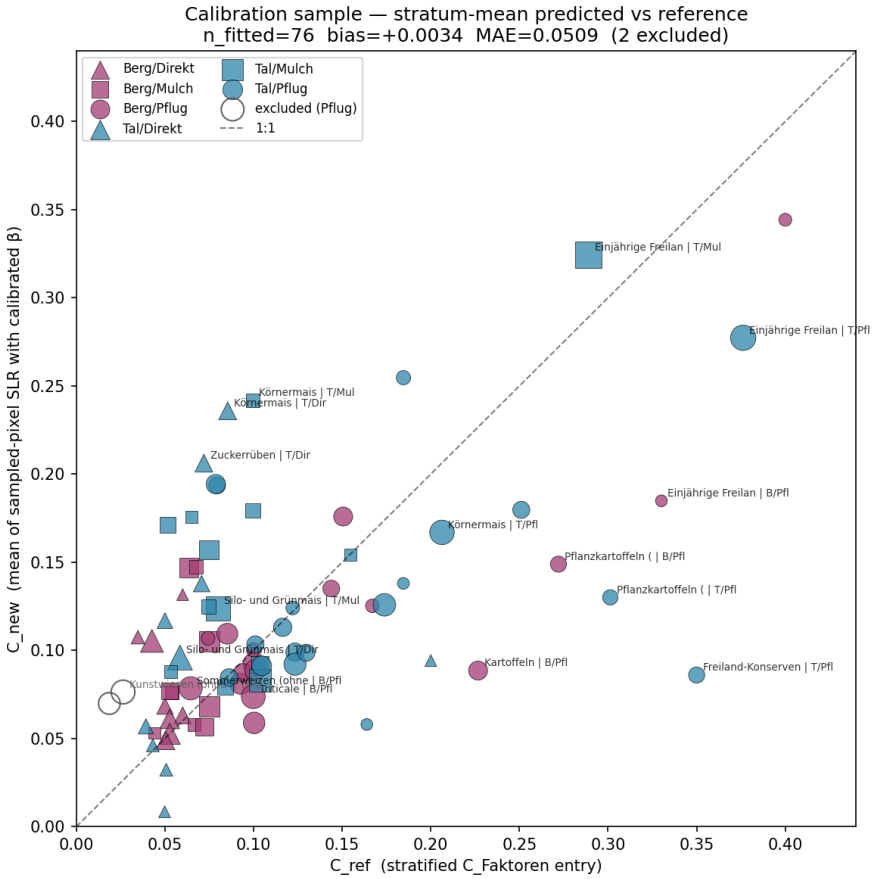


Figure 1: Predicted vs Reference C-factor rolled up per stratum using best calibrated model.

Keys that are specific to training only (`model_kind`, `results_folder`, `model_params`) are ignored by tune.

`tune_cfactor_ml.py` performs a cross-validation search to select the MLP architecture and regularisation strength, and to check whether an MLP buys anything over a linear (Ridge) model. By default (`cv_group: 'stratum'`), the CV folds are grouped at the crop×region×tillage stratum level, so the model is evaluated on strata it has never seen during training – this is meaningful because all pixels of a crop/stratum share the same broadcast label.

Mirroring a `'stratified'` train split (default) If `'split_strategy'` is set to `'stratified'`, tune builds a `StratifiedParcelKFold` that mirrors the per-stratum parcel holdout used by `train_cfactor_ml.py` (Section 6.4): parcels (`poly_id`) are distributed across folds within each stratum so every multi-parcel stratum sits in both the train and test side of the folds it appears in, and single-parcel strata are pinned to train in every fold. `cv_group` is ignored in this mode. Because no whole stratum is held out, this CV measures per-stratum *coverage* rather than generalisation to unseen strata. The train-vs-CV-gap interpretation in `tune_summary.txt`'s "how to read this" section does not apply in this mode.

The hyperparameter search includes:

- **Baselines**: DummyRegressor (global mean), Ridge, and HistGradientBoostingRegressor.
- **MLP grid**: combinations of `hidden_layer_sizes` and `alpha` (ℓ_2 weight decay).
- **Validation curve**: CV vs train MAE across α for the best architecture.

Outputs go to `tune_results_folder` (default `'calibration_analysis_tune/'`): `tune_cv_results.csv`, `tune_baselines.csv`, `tune_feature_importance.csv` (permutation importance from a HistGradientBoostingRegressor on one CV fold), `tune_best_config.json`, `tune_summary.txt`, and `plots/tune_validation_curve_alpha.png`, `plots/tune_model_comparison.png`.

6.4 Model training

Run:

```
python train_cfactor_ml.py
```

`train_cfactor_ml.py` is a self-contained training script with its own `CONFIG` at the top of the file. It:

1. Assembles the training table via `assemble_training_table` (reads the gapfilled parquet, loads EI, joins reference stratified C-factors). Rows whose `lnf_code` is in `exclude_calibration_lnf_codes`, or for which no reference C_{ref} exists, are kept (not dropped) and flagged `excluded=True`: they never enter training or test metrics but are still predicted, giving zero-shot inference for crops the model never trained on.
2. Builds the estimator: a `StandardScaler` + `WeightedMLPRegressor` or `Ridge` pipeline. The hyperparameters are read from `tune_best_config.json` unless overridden in `model_params`.
3. Trains with optional sample weighting (mirrors the β calibration's area-weighted loss).
4. Evaluates and saves outputs.

WeightedMLPRegressor (`weighted_mlp.py`)
A small PyTorch MLP (stacked linear layers with a configurable activation, sized by `hidden_layer_sizes`) wrapped as an `sklearn`-compatible estimator (`BaseEstimator`, `RegressorMixin`) – it is *not* a subclass of `sklearn.neural_network.MLPRegressor`, but mirrors its constructor arguments and `fit`/`predict` interface so it is a drop-in replacement. `fit(X, y, sample_weight=...)` accepts per-sample weights (standard `MLPRegressor` does not): weights are normalised to mean 1 and applied directly in the MSE loss ($\sum_i w_i(\hat{y}_i - y_i)^2$), so each example contributes proportionally to its weight while keeping the loss on the same scale as the unweighted case. Training uses `AdamW`, with `alpha` passed as its (decoupled) `weight_decay` – mathematically different from sklearn's L2 penalty, so a tuned `alpha` may not transfer exactly between the two implementations. Early stopping holds out `validation_fraction` of the (weighted) data and stops after `n_iter_no_change` epochs without improvement in weighted validation loss, restoring the best-validation-loss weights.

6.4.1 Train/test split strategies

Three strategies are available via 'split_strategy' in the training script `CONFIG`.

`'parcel'` A single `GroupShuffleSplit` on `poly_id`: parcels (not individual pixel-years) are randomly assigned to train (70%) or test (30%). This avoids data leakage within a parcel but strata may land entirely in train or entirely in test.

`'stratified'` (default) Per-stratum parcel holdout. Within each stratum (crop, or crop×region×tillage when stratified), parcels are split via `StratifiedGroupKFold` so that every stratum with ≥ 2 distinct parcels has at least one parcel on each side – no parcel, and therefore no pixel, crosses the split. Strata with only a single parcel cannot be split and

go to train only (still learned from, never scored in test). Because labels are broadcast per stratum, a multi-parcel stratum's C_{ref} is then seen during training too, so the test metric reflects per-stratum *coverage* rather than generalisation to unseen strata. The fitted pipeline saved to `nn_model.joblib` is the model trained on the train split only. Given the C-factors are only computed for previous years, such a training strategy is compatible with an operational approach.

`'loyo'` Leave-One-Year-Out cross-validation: for each calendar year, the model is trained on all other (trainable) years and evaluated on the held-out year. Out-of-fold predictions cover all trainable data, plus year-matched zero-shot predictions for excluded rows. After the LOYO loop, a separate *final model* is fitted on all trainable data across all years – this final model is what is saved to `nn_model.joblib`, with its in-sample predictions stored in the `C_pred_final` column. Due to the highly imbalanced dataset over the years, this strategy is not recommended.

6.5 Outputs

`train_cfactor_ml.py` writes the following to `results_folder`:

File	Content
`nn_model.joblib`	Fitted pipeline + metadata (model_kind, split_strategy, feature/join columns, effective hyperparameters, n_steps).
`nn_training_features.csv`	Full per-pixel feature+target table, incl. `split` and `excluded` columns.
`nn_predictions_per_pixel.csv`	Per-pixel `C_ref`, `C_pred` (plus `C_pred_final` in LOYO mode), `split`, `excluded`.
`nn_predictions_per_crop.csv`	Predictions averaged to (crop) or (crop, region, tillage) group.
`nn_metrics.json`	Metrics; structure depends on `split_strategy`: pixel/crop-level train+test for `'parcel'` and `'stratified'` (the latter also reports `n_single_parcel_strata_train_only`); `loyo_aggregate`, `loyo_per_year` and `final_model_insample` for `'loyo'`.
`nn_scatter.png`	Predicted vs reference at group level (trainable groups only).
`nn_loss_curve.png`	MLP training loss curve (`model_kind='mlp'` only).
`nn_ridge_coefficients.png`	Ridge coefficient magnitude per timestep, faceted by PV/NPV/EI channel (`model_kind='ridge'` only).

Excluded (inference-only) rows Pixels whose `lnf_code` is in `exclude_calibration_lnf_codes`, or that have no matching reference C_{ref}, appear in `nn_predictions_per_pixel.csv` with `excluded=True` and `split='excluded'`. They never contribute to training or to train/test metrics, but every fold and the final model still predict them, giving zero-shot C-factor estimates for crops the model never trained on.

Post-hoc analysis `analyse_ml_sample.py` reads the above outputs and auto-detects the split mode from the `split` column (4-digit-year values → LOYO; `train`/`test` → parcel/stratified). It writes `nn_summary.txt`; `nn_per_crop.csv` or `nn_per_stratum.csv` (one block per split); `nn_by_region.csv`, `nn_by_tillage.csv` and `nn_by_c_magnitude.csv` (stratified mode); and `nn_excluded_{crop,stratum}s.csv` if any rows were excluded. Plots go to `results_folder/plots/`, drawn by default on all samples (`'analyse_split': 'all'`, can be

changed to `'train'` or `'test'` set): per-crop/per-stratum scatter, strengths/weaknesses, box and interannual plots, plus mode-specific diagnostics – `nn_train_vs_test_scatter.png` (parcel/s-tratified, over-fit check) or `nn_loyo_per_year_scatter.png` and `nn_loyo_vs_final_scatter.png` (LOYO) – and `nn_excluded_crops_scatter.png` / `nn_excluded_crops_box.png` for excluded crops. Set `results_dir` (and the input paths) in the CONFIG block and run:

```
python analyse_ml_sample.py
```

6.6 Best model

The ML estimator (Section 6) is fitted on the same per-pixel FC+EI feature table as the empirical calibration, against the stratified (Tal/Berg × tillage) C-factor targets. Model selection (`tune_cfactor_ml.py`) compares candidate estimators under 2-fold *grouped* cross-validation that mirrors the stratified-parcel train/test split (Section 6.3). This means folds hold out whole crops/strata, so the score measures generalisation to crop×strata the model has not fitted. The candidates are:

- **Dummy** (global mean) — the floor any model must beat.
- **Ridge** — linear reference: is a non-linear model buying anything?
- **HistGradientBoosting** — strong tabular tree baseline.
- **MLP** — grid over `hidden_layer_sizes`×`alpha`.

As on the empirical side, tuning was run with and without the ley class (LNF 601, 602). The primary metric is the **group-mean MAE** under grouped CV (predictions averaged to the crop/stratum level before scoring), which is directly comparable to the empirical calibration's cross-crop MAE. Removing ley lowers the score for every model and leaves 76 independent groups (1,375 pixel rows) versus 78 (1,466 rows) with ley retained.

	Group-mean MAE (grouped CV)	
Model	with ley	no ley
Dummy (mean)	0.0608	0.0587
Ridge	0.0472	0.0464
HistGradientBoosting	**0.0383**	0.0379
MLP $(32, 16)$	0.0396	**0.0363**

Table 2: Grouped-CV model comparison (lower is better); best per column in bold. The MLP uses $\alpha = 0.01$ with ley and $\alpha = 0.1$ without. The selected estimator is the no-ley MLP at 0.0363 ± 0.0017.

The MLP beats Ridge in both runs, so there is non-linear signal at the group level worth the extra capacity; HistGradientBoosting is a close tree baseline — marginally ahead with ley, marginally behind without. Excluding ley improves every model and gives the best overall result, the **MLP** $(32, 16)$**,** $\alpha = 0.1$ at a group-mean CV MAE of 0.0363 ± 0.0017, consistent with the empirical no-ley choice (Section 5.7). Permutation importance is dominated by a single spring PV feature (`pv_t030`), in line with peak green cover being the main control on the seasonal C-factor.

The chosen hyper-parameters are recorded in `tune_best_config.json`; the pipeline refit on all data is saved as `nn_model.joblib` in `calibration_analysis_mlp_strat_noley/`. This is the artefact loaded by `compute_cfactor_pixels_ml.py` (Section 9).

7 Comparison: Empirical vs ML

Run:

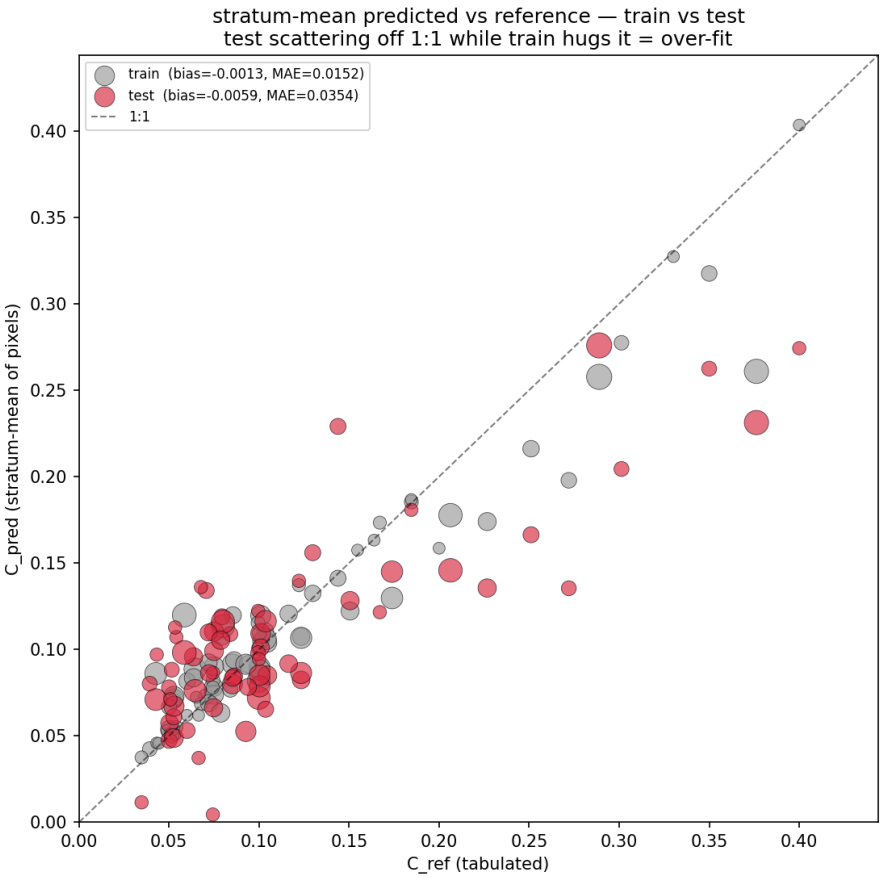


Figure 2: Stratum level train vs test performance.

```
python compare_beta_vs_ml.py
```

`compare_beta_vs_ml.py` inner-joins the β pipeline's per-pixel predictions (`calibration_results_stratified_per_pixel.csv`) with the ML pipeline's (`nn_predictions_per_pixel.cs` on `(lnf_code, yr, poly_id)`, aggregates to the stratum level (`lnf_code, region, tillage`), and reports the area-weighted stratum MAE for both models side by side. The empirical β side is always in-sample: with 1–2 degrees of freedom it cannot meaningfully overfit, so its in-sample MAE $\approx$ its generalisation MAE.

7.1 Which ML predictions are compared

`'ml_predictions_source'` selects which rows of the ML per-pixel CSV enter the comparison:

- `'test'` (default): held-out test rows only (`split == 'test'`), for a `train_cfactor_ml.py` run with `split_strategy 'parcel'` or `'stratified'` – no parcel/pixel leakage. The exact sub-strategy is read from the sibling `nn_metrics.json`'s `split_strategy` field, since the per-pixel `split` column alone cannot distinguish `'parcel'` from `'stratified'`.
- `'loyo'`: all rows, for a `'loyo'` training run – every row is an out-of-fold prediction.
- `'final'`: the `C_pred_final` column (the deployed, in-sample model; only written in LOYO mode) – an optimistic ceiling, sanity-check only.
- `'all'`: all rows pooled (train+test) – optimistic, not an honest comparison.

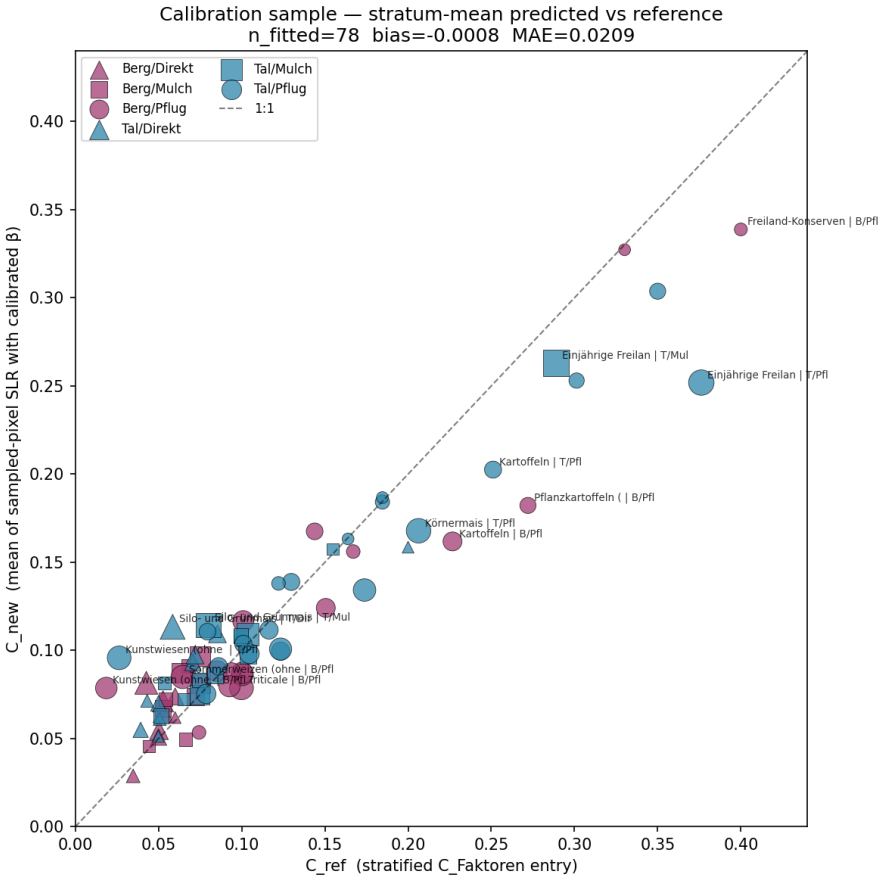


Figure 3: Predicted vs Reference C-factor rolled up per stratum using best ML model, predicted on whole dataset (including train).

- `'auto'`: `'loyo'` if the `split` column holds 4-digit years, else `'test'`.

In every mode, `'excluded'` rows (config-excluded crops or crops with no reference C_{ref}) are dropped before the comparison. Because the inner-join restricts to the ML rows selected above, both models are scored on exactly the same pixels and strata: if ML wins under `'test'` or `'loyo'`, the deployed ML model is at least as good as β.

7.2 Weighting and outputs

Stratum weight $= A_c \times n_{\text{pixels,stratum}}/n_{\text{pixels,crop}}$, normalised over the common (post-join) stratum set – the same scheme as the β calibration's stratum weighting and the ML pipeline's `compute_sample_weights` (Section 6.4). A_c (Swiss arable area per crop) is read from `calibration_results_str`

Outputs go to `'out_dir'` (default `compare/`):

File	Content
`compare_summary.txt`	Headline area-weighted stratum MAE/bias for β and ML, stratum-level winner counts, per-year and top-10-crop breakdowns, and a "how to read this" note tailored to the chosen ML source/split mode.
`compare_per_stratum.csv`	Per-(crop, region, tillage): both predictions, residuals, weight, winner.
`compare_per_crop.csv`	Rolled up to crop (mean over its strata).
`compare_per_year.csv`	Year-wise area-weighted MAE (only written if the joined data spans multiple years).
`plots/compare_scatter.png`	Side-by-side β vs ML scatter (stratum means vs C_{ref}), worst-5 strata labelled per panel.
`plots/compare_paired.png`	Per-stratum β residual vs ML residual, coloured by region, marker by tillage, sized by weight – shows whether ML's wins are concentrated in specific strata or the two models share the same errors.
`plots/compare_per_crop_bars.png`	Top-N crops by area: paired |bias| bars and per-crop winner.
`plots/compare_per_year.png`	Year-wise MAE for β vs ML (if multi-year).

7.3 Results

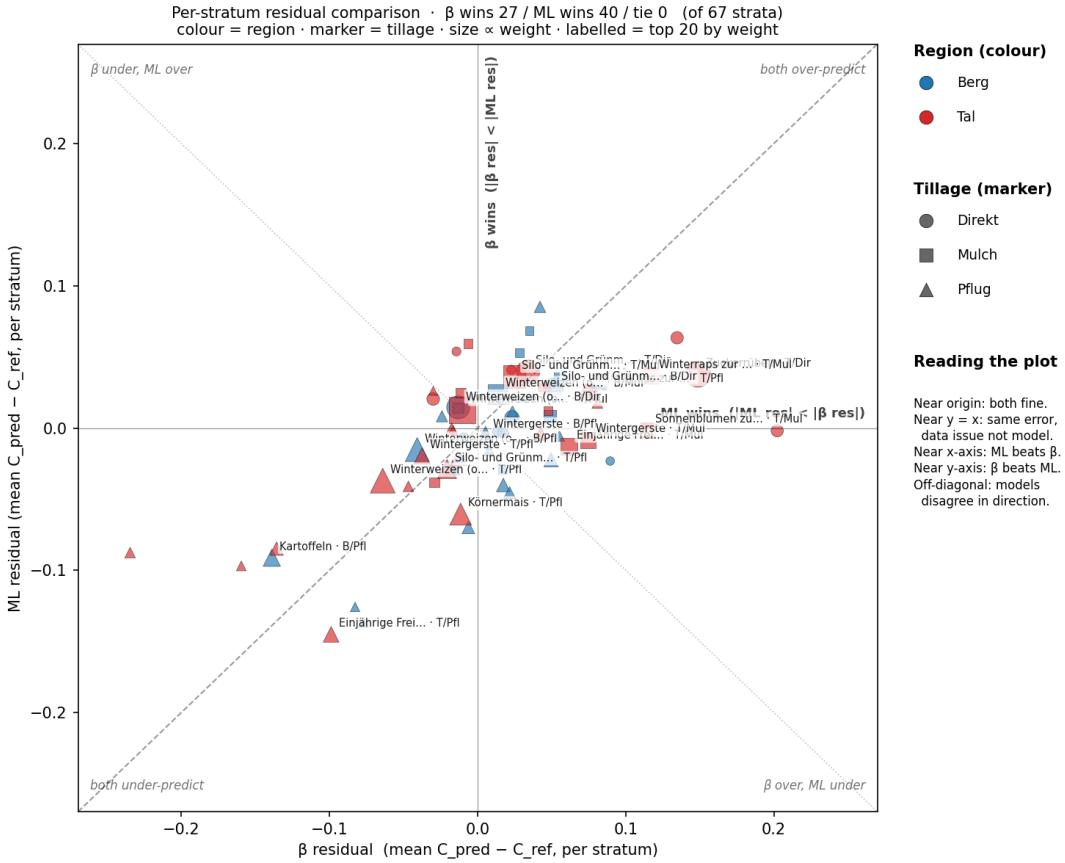


Figure 4: Comparison of ML residuals and calibrated function residuals compared to reference C-factors, on the sampled dataset.

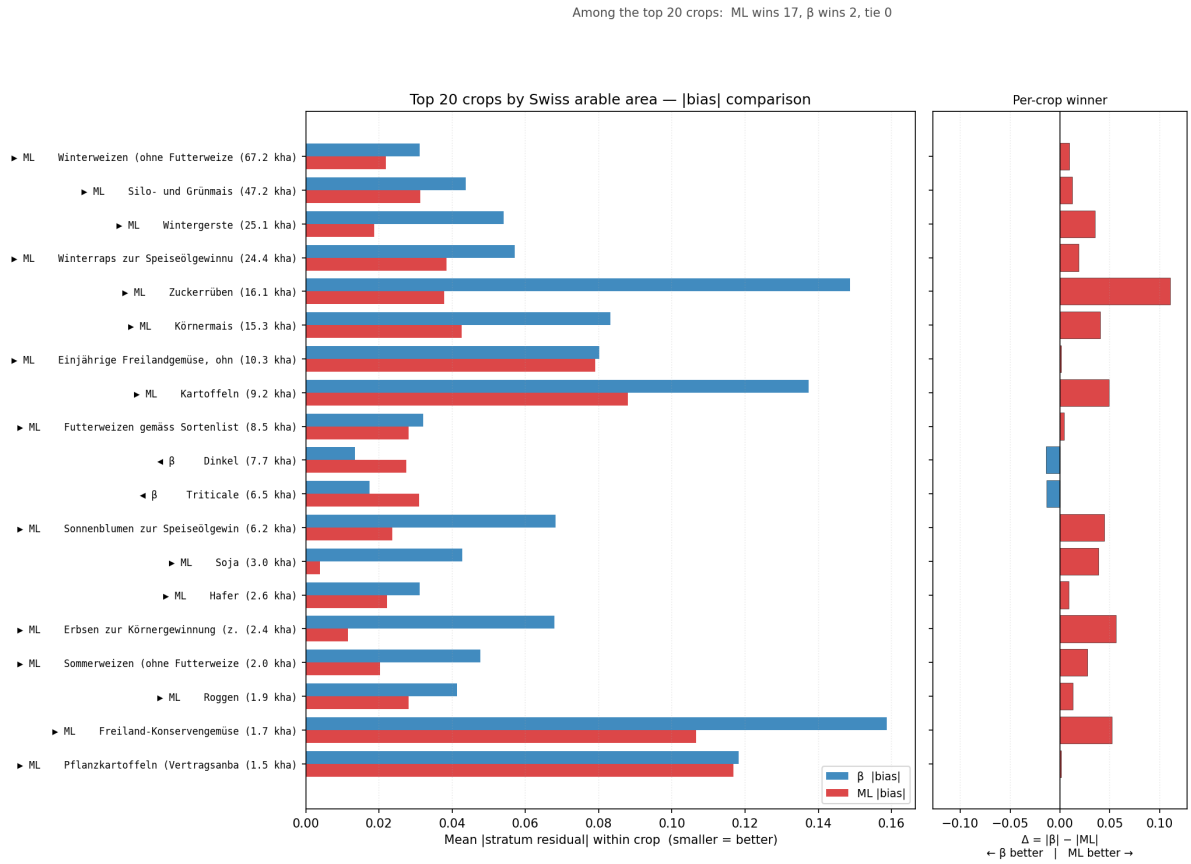


Figure 5: Comparison of ML and calibrated function approach for the top 20 crops (per area).

8 Running the Full Pipeline

8.1 Prerequisites

1. Python 3.12 environment with the standard scientific stack (`numpy`, `pandas`, `geopandas`, `xarray`, `rioxarray`, `scikit-learn`, `torch`, `joblib`, `pyarrow`, `scipy`, `matplotlib`).
2. Scripts configured correctly (Section 10).
3. Network access to `~/mnt/eo-nas1` (NAS) and files/folders mentioned in 2.

8.2 Quickstart

Step 1: Run the empirical calibration pipeline

```
# Full pipeline: sampling + calibration
python main.py

# Skip sampling if samples_data_gpr.parquet already exists:
python main.py --skip-sampling
```

Step 2: Check sampled data (optional)

```
python check_sampled.py
# -> sample_analysis/ directory with diagnostic plots
```

Step 3: Analyse calibration results (optional)

```
python analyse_calibration_sample.py
# -> calibration_analysis_noley/plots/ with detailed diagnostics
```

Step 4: Tune the ML model

```
python tune_cfactor_ml.py
# -> calibration_analysis_tune_noley/ with CV results and best config
```

Step 5: Train the ML model

```
python train_cfactor_ml.py
# -> calibration_analysis_mlp_loyo_noley/ with model + predictions
```

Step 6: Analyse ML results (optional)

```
python analyse_ml_sample.py
# -> calibration_analysis_mlp_loyo_noley/plots/
```

Step 7: Compare empirical vs ML

```
python compare_beta_vs_ml.py
# -> compare_Bnoley_MLloyonoley/ with comparison plots and summary
```

8.3 Expected runtimes

Step	Approx. runtime
Sampling (AGIS strategy, 10k samples, 6 years)	10–30 min
S2 extraction + FC prediction	2–4 h
GPR gap-fill (regular, 2 workers)	2–8 h
Calibration (β fit)	5 min
ML tuning	30 min
ML training	5 min

9 Operational Application

Inference window and the agricultural year

The two methods differ in how strictly they are bound to the agricultural year (1 July of *year* − 1 to 30 June of *year*):

- **Empirical (β) method.** The reduction $C = \sum_t \mathrm{SLR}(t)\,\mathrm{EI}(t) / \sum_t \mathrm{EI}(t)$ is just an EI-weighted integral and is therefore mathematically valid over *any* time window. Here it is applied over the agricultural year for consistency, but it can be run on an arbitrary timeframe if needed.
- **ML method.** The model is *trained* on features aggregated over the agricultural year (1 July of *year* − 1 to 30 June of *year*), so it is only meaningful when applied over that same July–June window. A different window feeds the network a feature vector it never saw in training and provides a prediction it is not meant to do.

The way C-factors were *previously* derived (MAUS) also keyed the value to the main crop

(Hauptkultur) present in each field, taken from the annual LNF declarations — which themselves follow the agricultural year more closely than the calendar year. Aligning the Sentinel-2 inference window with that crop calendar keeps the new per-pixel product directly comparable to the existing one.

Year-label convention: `-years 2022` denotes the agricultural year ending 30 June 2022, not a calendar year.

Once a model has been established (either the calibrated β from Section 5 or the trained ML pipeline from Section 6), it is applied to *every* Sentinel-2 pixel inside agricultural fields, to produce a spatially explicit per-pixel C-factor product. Two scripts implement this, sharing the same input path and differing only in the final reduction:

- `compute_cfactor_pixels.py` — empirical product. Loads β from `beta.json` and computes $C = \sum_t \mathrm{SLR}(t)\,\mathrm{EI}(t) / \sum_t \mathrm{EI}(t)$ with $\mathrm{SLR} = \exp(-\beta \cdot \mathrm{FC})$.
- `compute_cfactor_pixels_ml.py` — ML product. Loads the trained pipeline (`nn_model.joblib`) and predicts C directly from the per-pixel $(\mathrm{PV}, \mathrm{NPV}, \mathrm{EI})_t$ feature vector.

Both run the same three-stage pipeline.

- Stage A is parallel over spatial tiles: it rasterises the LNF field polygons onto the S2 grid, loads the raw S2 bands plus the `SCL`/`mask` bands for the agricultural year (1 July of $year-1$ to 30 June of $year$), and cleans cloud/shadow/snow/cirrus per pixel-date.
- Stage A½ predicts FC for all clean observations (GPU-optional).
- Stage B is parallel over fields: a GP gap-fills FC onto the regular `grid_step_days` grid at every field pixel, EI is joined climatologically, and the per-pixel C is reduced (β reduction or ML prediction). Splitting tiles (Stage A) from fields (Stage B) ensures fields spanning a tile boundary are never cut.

9.1 Prerequisites

In addition to the input datasets of Section 3 (raw S2, DLR soil groups, FC models, LNF `.gpkg`, EI grid), the operational step needs the artefact produced by the chosen model:

- empirical: `beta_stratified.json` (or `beta.json`), written by the calibration in Section 5;
- ML: `nn_model.joblib`, written by `train_cfactor_ml.py` (Section 6). The `grid_step_days` used here *must* match the value used in training, since it sets the number of time steps in the feature vector.

The data paths are read from the `CONFIG` dict at the top of each script (Section 10); `-region`, `-years`, `-model`, `-gpu` and `-geotiff` can be set on the command line and override `CONFIG`.

9.2 Running for a region

A region is any OGR-readable vector (`.shp`, `.gpkg`, ...) in any CRS; only tiles and fields intersecting it are processed. This is the recommended way to start, as cost scales with the number of fields.

Empirical (β) product

```
# One year over a region, FC on GPU, also write per-tile GeoTIFFs
python compute_cfactor_pixels.py --region seeland.gpkg --years 2022 --
    gpu --geotiff
```

```
# Several years at once
python compute_cfactor_pixels.py --region seeland.gpkg --years 2021
   2022 2023
```

ML product

```
# Point --model at the joblib bundle from train_cfactor_ml.py
python compute_cfactor_pixels_ml.py --region seeland.gpkg --years 2022
    --gpu \
    --model calibration_analysis_mlp/nn_model.joblib
```

9.3 Running for all of Switzerland

Omitting `-region` processes every available S2 tile, i.e. the whole country. The scripts print a warning to confirm this is intended:

```
# Whole country, single year (long-running: see runtimes below)
python compute_cfactor_pixels.py --years 2022 --gpu
python compute_cfactor_pixels_ml.py --years 2022 --gpu \
    --model calibration_analysis_mlp/nn_model.joblib
```

A country-wide run is dominated by one GP fit per field (Stage B) and the FC inference in Stage A½; the `n_workers_io` and `n_workers_gp` keys control parallelism for the two stages. Restrict to specific crops with the `lnf_codes` `CONFIG` key if a full crop run is not needed.

9.4 Outputs

Both scripts write to `output_dir` (default `output/cfactor_pixels` resp. `output/cfactor_pixels_ml`):

- **Per-pixel Parquet**, hive-partitioned as `year=YYYY/lnf_code=NNN/part-*.parquet`, with columns `poly_id, lnf_code, x, y, year, c_factor, n_clean_obs_field, n_grid_pts`.
- **Per-field summary** (`write_field_summary`, default `True`): `cfactor_fields_YYYY.parquet` with the area-mean, std, median and pixel count of C per field. This matches the granularity of the existing R erosion product, which assigns one C-factor per Nutzungsfläche, and is the natural quantity for the comparison in Section 12.
- **GeoTIFF** (optional, `-geotiff`): one 10 m C-factor raster per tile-block.

10 Configuration Reference

All parameters are set in the `CONFIG` dict at the top of `main.py` (empirical pipeline), `train_cfactor_ml.py` (ML pipeline), and `compute_cfactor_pixels.py` / `compute_cfactor_pixels_ml.py` (operational per-pixel inference, Section 9). They share the same data-path keys; the operational keys are listed at the end of the table.

Table 3: Main `CONFIG` keys.

Key	Default	Description
Sampling and gap-fill		
`lnf_labels_path`	xlsx path	LNF classification spreadsheet.

(continued)

Key	Default	Description
lnf_dir	dir path	Directory of annual LNF .gpkg files.
top_crops	None	Explicit crop list; None = auto from area columns.
lnf_ignore_codes	[...]	LNF codes always excluded.
grassland_codes	[601]	LNF codes treated as grassland despite lv3 label.
tot_samples	10000	Total target sample count.
years	2019–2024	Years to sample from.
top_crops_area_years	[2022, 2023, 2024]	Years used to rank top crops by area.
sampling_strategy	'agis'	'random' or 'agis'.
agis_rules	all three rules	Tuple of AGIS selection rules.
agis_dominant_share	0.70	Minimum crop share for dominant_crop rule.
group_crops_by_cfactor	True	Pool crops with identical C-factor vectors.
c_factor_provenance_path	xlsx path	Provenance table (to drop estimated-only codes).
fc_precompute	False	Use pre-computed FC zarrs if available.
cirrus_thresh	500	Blue band threshold for extra cirrus masking.
max_missing_frac	0.05	Max fraction of masked pixels per date per field.
drop_fraction_threshold	0.70	Drop field if this fraction of dates masked.
gapfill_method	'regular'	'regular' (grid_step_days step) or 'irregular' (only fill gaps ≥max_gap_days days).
grid_step_days	10	Grid cadence for regular gap-fill (days).
max_gap_days	15	Gap threshold for irregular gap-fill (days).
n_jobs	2	Parallel workers for GPR.
Calibration		
ei_path	parquet path	Gridded EI parquet.
c_factor_table_path	CSV path	Reference C-factor table.
beta_bounds	(1e-4, 0.1)	Search bounds for β.
calibration_mode	'single'	'single' or 'two_beta'.
area_weight_loss	True	Weight loss by Swiss arable area per crop.
area_years	[2022,2023,2024]	Years for area weight computation.
exclude_calibration_lnf_codes	[601,602]	Codes excluded from calibration target.
stratified_calibration	True	Use per-stratum C-factors as target.
grenze_tal_berg	600	Altitude cutoff for Tal/Berg (metres).

(continued)

Key	Default	Description
standardansaatverfahren	'Pflug'	Default tillage when REB entry absent.
tillage_assignment	'stochastic'	How to resolve mixed-tillage farm-years.
nutzung_csv	CSV path	Field level crop type and area.
ressourceneffizienz_csv	CSV path	Farm level soil preparation data
results_folder	dir name	Output directory for calibration results.
ML training (in train_cfactor_ml.py)		
model_kind	'mlp'	'mlp' or 'ridge'.
split_strategy	'stratified'	'parcel', 'stratified', or 'loyo'.
test_fraction	0.3	Test fraction ('parcel' and 'stratified' splits).
area_weight_loss	True	Mirror area-weighted loss in training.
tune_best_config_path	json path	Tuner output; hyperparameters are read from here.
model_params	{}	Per-model overrides (e.g. alpha).
Operational inference (compute_cfactor_pixels.py / _ml.py)		
beta_path	json path	Calibrated β (empirical script).
model_path	joblib path	Trained ML pipeline (ML script; or -model).
clip_cfactor	True	Clip ML predictions to $[0, 1]$ (ML script).
region_path	None	Region vector; None = whole country (or -region).
s2_dir	dir path	Raw Sentinel-2 zarr archives.
soil_dir	dir path	DLR soil-group zarrs (FC model selection).
lnf_dir	dir path	Annual LNF .gpkg files.
ei_path	parquet path	Climatological EI grid.
output_dir	dir path	Output root for the per-pixel product.
years	[2022]	Agricultural years to process (or -years).
lnf_codes	None	Restrict to these LNF codes; None = all crops.
lnf_ignore	[]	LNF codes to always exclude.
n_workers_io	6	Stage A (tile) workers.
n_workers_gp	6	Stage B (field) workers.
fc_model_dir	dir path	FC ensemble models.
use_gpu	False	GPU for FC prediction if available (or -gpu).
clean_mode	'per_pixel'	'per_pixel' (drop masked pixel-dates) or 'field_date'.
grid_step_days	10	GP grid cadence; must match ML training.
max_train_points	1000	Max clean obs per field GP fit.

(continued)

Key	Default	Description
`min_clean_obs`	3	Min clean obs to gap-fill a field.
`ei_half_pixel_shift`	`True`	Align S2 and EI pixels.
`write_geotiff`	`False`	Write per-tile 10 m GeoTIFFs (or `-geotiff`).
`write_field_summary`	`True`	Write per-field area-mean C-factor parquet.

11 Design Decisions and Rationale

Regular gap-fill The irregular mode keeps real observations but produces time series of different lengths, making ML feature construction non-trivial. The regular mode discards the original observations (they become training data for the GP rather than outputs), but produces a uniform 37-step vector that is directly usable as a fixed-length feature vector. For the β calibration both modes work equally well; for ML the regular mode is required.

ALR transform for GPR Fractional cover is a three-component compositional variable (PV + NPV + Soil = 1, each in $[0, 1]$). A standard GP on any single component could predict values outside $[0, 1]$ or fractions that do not sum to unity. The ALR transform maps the simplex to $\mathbb{R}^2$ where GP assumptions are valid; the inverse ALR automatically satisfies the compositional constraint.

Pixel-level FC One representative pixel per parcel is used (the `is_sample_pixel` flag) rather than averaging all field pixels. The GP is trained on all clean field pixels (spatial context) but predicts at the single target pixel. This design avoids within-field spatial averaging, which could blur edge effects, tile boundaries, and within-field variability. The goal is also to have a product operating at the pixel-level.

AGIS/BLW-informed sampling We need to know the soil preparation done to the fields, which is not reported in the LNF dataset. The AGIS farm-level rules select fields where the crop identity is most likely to be reliable: single-crop farms, or farms predominantly grassland with a single arable crop. This reduces label noise in the training dataset. An additional dataset from BLW with detailed farm x crop info is also used

Area-weighted loss Switzerland's arable area is dominated by a few crops (winter wheat, maize, rapeseed). An equal-weighted MAE across crops would give the same influence to a rare specialty crop (few hundred hectares) as to winter wheat (hundreds of thousands of hectares), potentially pulling β to fit the outlier at the expense of the dominant crops. Area weighting makes the calibrated model optimal for the country's actual crop distribution. The area weighting is also applied in the ML pipeline.

Stratified CV for ML tuning In an operational setting, a user would have access to all crop type data from the previous year(s), and could train on the year that also needs to be predicted. There is no real out of domain since we don not look into the future. Therefore, instead of doing a LOYO split, the data is split intro train/test while ensuring that strata (crop x tillage type x altitude) appear in both sets. If there is not enough data, the strata will only appear in train. This stategy therefore assess the coverage of the model to unseen pixels on the same stratum and represents an upper boundary of the performance.

12 Case Study: Seeland

Three C-factor products are compared:

1. **Previously calculated (reference).** The C-factor as assigned by the existing Swiss erosion-risk pipeline, stored at `/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected/Resultate/Erosionsrisiko`.
2. **Calibrated (β).** The empirical per-pixel product with the two-β no-ley calibration (Section 5.7).
3. **ML.** The per-pixel product with the selected no-ley MLP (Section 6.6).

Both the calibrated model and the ML model write a per-pixel C-factor and a per-field area-mean summary (`cfactor_fields_YYYY.parquet`). Because the legacy product is resolved at the field level, all three are compared at that common granularity: the per-pixel β and ML values are area-averaged to one C-factor per field, then placed alongside the reference value for the same field and year.

12.1 Study area

In order to compare the developed products, the models are applied on the Seeland district (canton of Bern), for the agricultural years 2021 and 2022 (i.e. July 2020–June 2021 and July 2021–June 2022; Section 9). An overview of the region is shown in Figure 6. The region is dominated by winter wheat and ley fields (Figure 7). Other common arable crops include vegetables, sugar beets, maize, potatoes and winter barley. There is little difference between the two years. According to Figure 8, plowing is the dominant soil preparation in the region, followed by mulching. Only a minority of the area uses direct seeding.

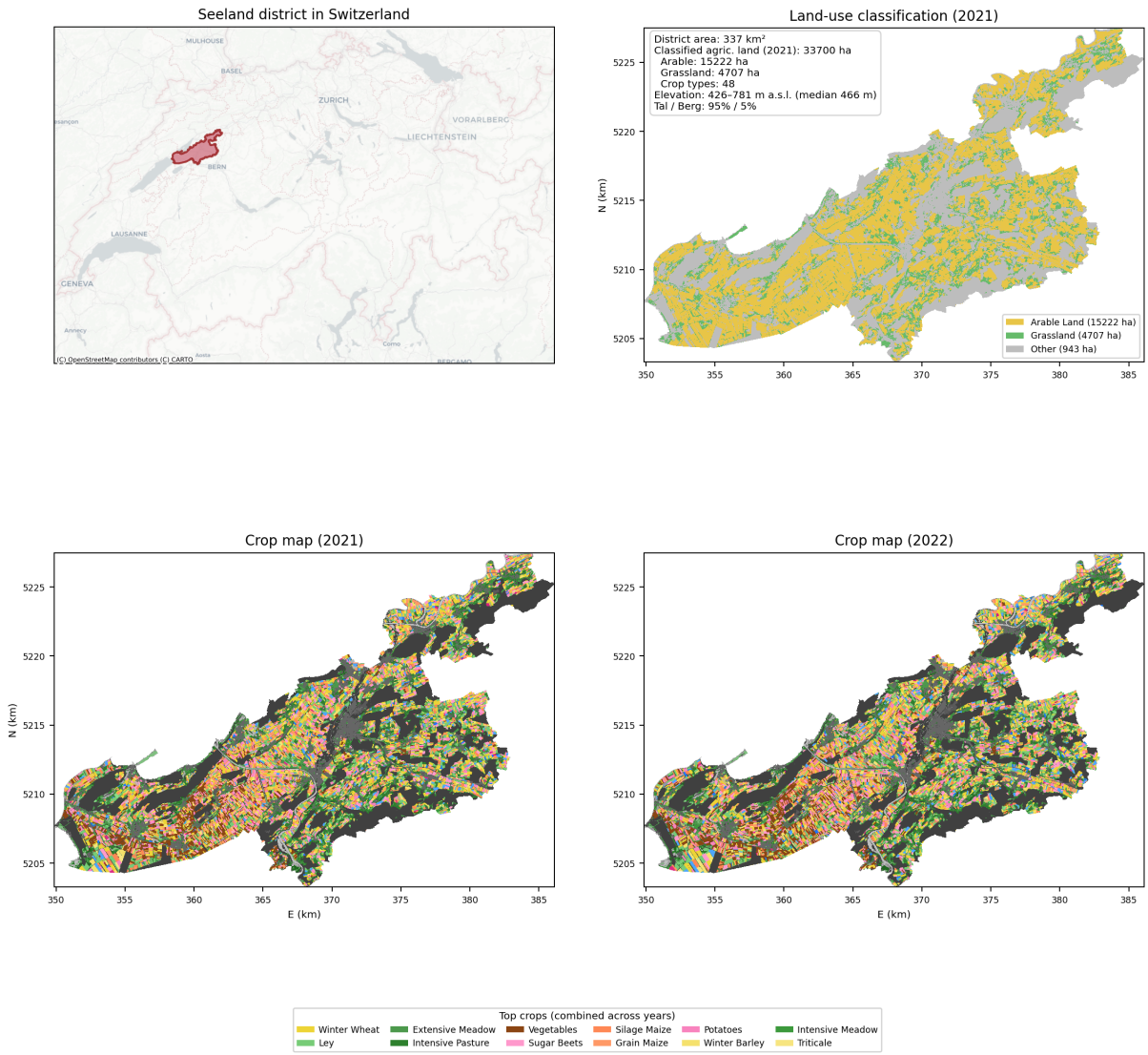


Figure 6: Overview of Seeland district. Top left: area highlighted in Switzerland, top right: areas of arable, grassland or other, bottom row: crop maps in 2021 (left) and 202 (right) predicted from Crop1990 in Seeland.

The two years were selected for having different weather patterns that could influence crop growth (Figure 9). The spring of 2022 was milder than 2021 (Figure 9a), most notably in May, potentially leading to an earlier emergence of crops and earlier soil cover. The 2020/21 agricultural year received considerably more precipitation during the late spring and early summer (Figure 9b), with over 350 mm falling in May–June 2021 alone, whereas the 2021/22 year was marked by an exceptionally wet July at its start (∼220 mm), corresponding to the post-harvest period where soils could be more exposed to erosive rainfall.

In 2020/21, the cool spring temperatures would slow crop emergence and early canopy development, so FC should rise relatively late. The abundant rainfall through May and June could produce a high and persistent FC peak that maintains good soil cover well into the summer. In 2021/22, the milder spring would favour earlier emergence and a faster initial rise in FC. The dry conditions through much of the spring, combined with warmer temperatures, would then limit canopy expansion and likely trigger earlier senescence, resulting in a lower and shorter-lived FC peak. The dry autumn of 2021 would also delay the establishment of winter cover, prolonging the period of low FC at the start of the year.

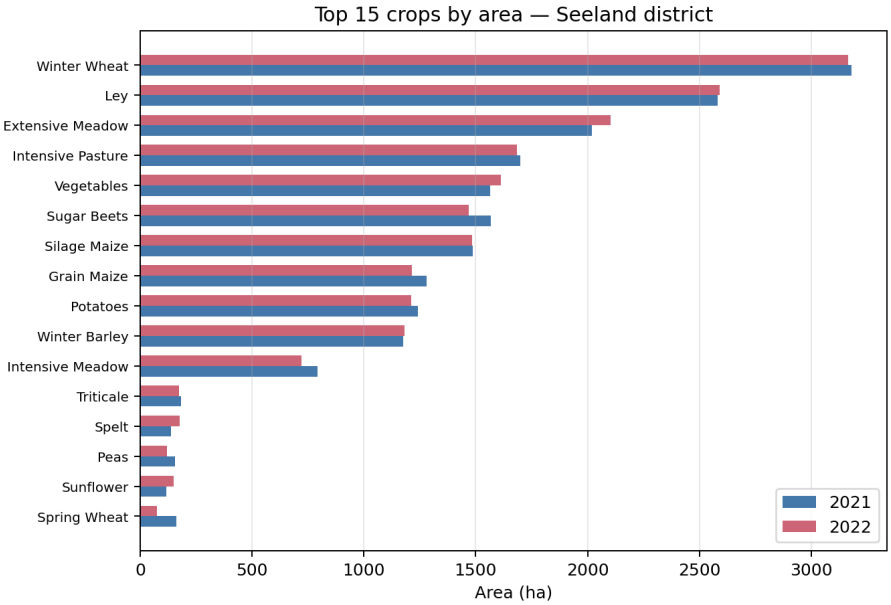


Figure 7: Distribution of the crops by area in 2021 and 2022 in Seeland district, extracted from Crop1990 predictions.

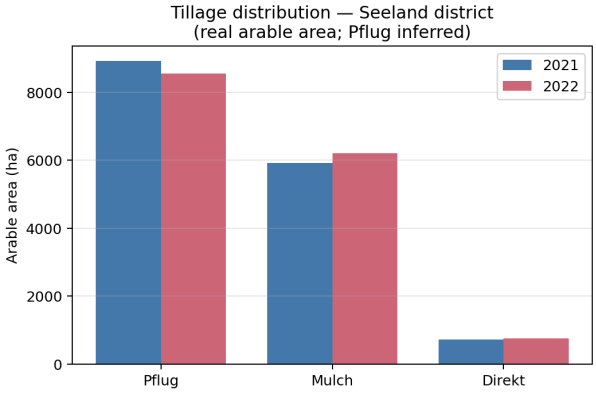
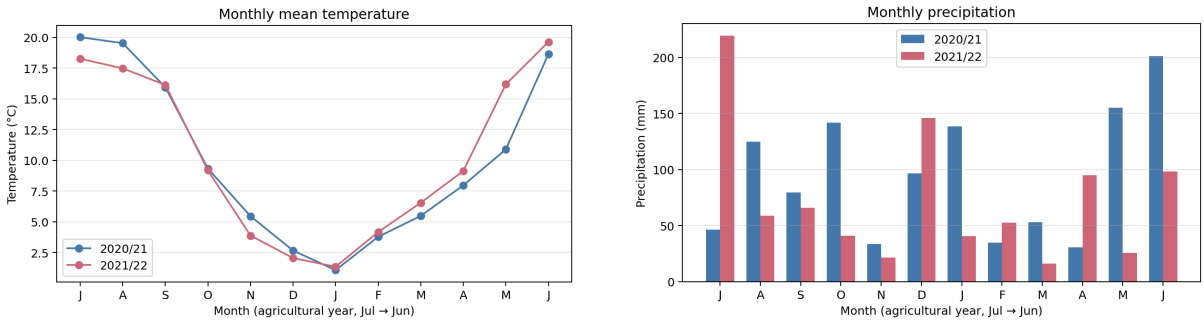


Figure 8: Distribution of tillage practices in 2021 and 2022, extracted from `BLW CSV`, see Section 3. 'Pflug': plow, 'Mulch': mulching, 'Direkt: direct seeding. Plowing is inferred where no method was assigned.



(a) Monthly average temperature

(b) Monthly total precipitation

Figure 9: MeteoSwiss derived statistics on the weather in Seeland in growing seasons 2021 and 2022.

Figures 10 shows the average timeseries of fractional cover of photosynthetic vegetation (PV), non-photosynthetic vegetation (NPV) and their sum (FC total) in agricultural years 2021 and 2022 in the district. We can see slight differences such as higher PV in spring for 2022 and higher NPV in July/August at the start of the agricultural year 2022.

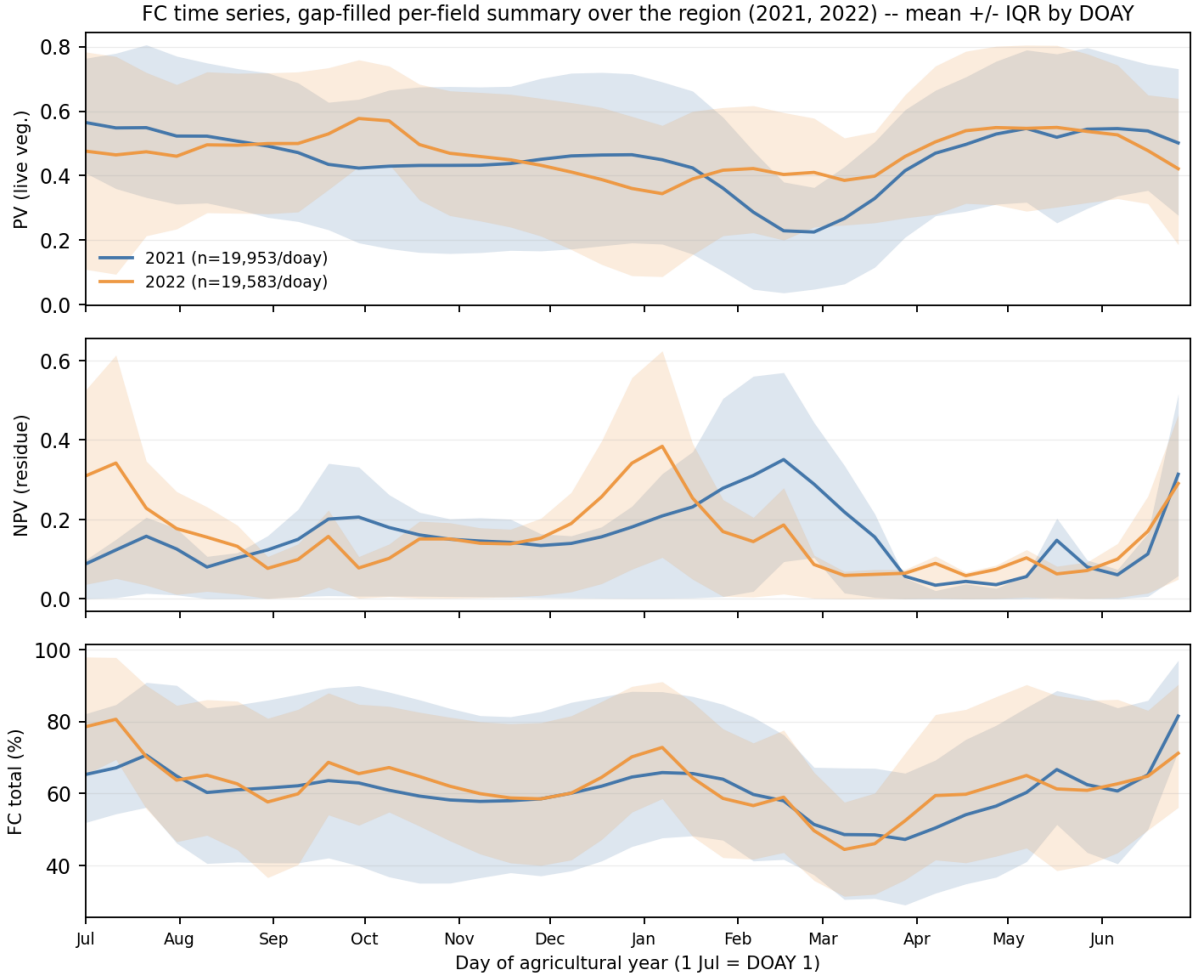


Figure 10: Average timeseries of fractional cover of photosynthetic vegetation (PV), non-photosynthetic vegetation (NPV) and their sum (FC total) in agricultural years 2021 and 2022 (DOAY: day of agricultural year) in Seeland.

We can see in Figure 11 the breakdown per crop. Major differences between the yeras can be observed for vegetables (Einjährige Freilandgemüse), maize (Körnermais, Silomais), sugarbeet (Zuckerrüben) and potatoes (Kartoffeln). The timeseries also varies depending on the crop: grasslands have high cover throughout the year, potato and sugarbeet fields are more bare in spring, while winter barley fields are already covered at that time of the year.

The erosivity index over the months is shown in Figure 12. Given that most erosive rainfall events occur in summer, the year 2021/22 could exhibit higher C-factors considering the shorter FC peak in summer compared to 2020/21 where the soil cover should be high in summer.

12.2 Within year product comparison

We start by comparing the products applied on the same year, to determine where differences occur and what could be driving them. A per-crop comparison using the 3 different approaches in shown in Figure 13 for each year. In 2021, the empirical model produces C-factors that

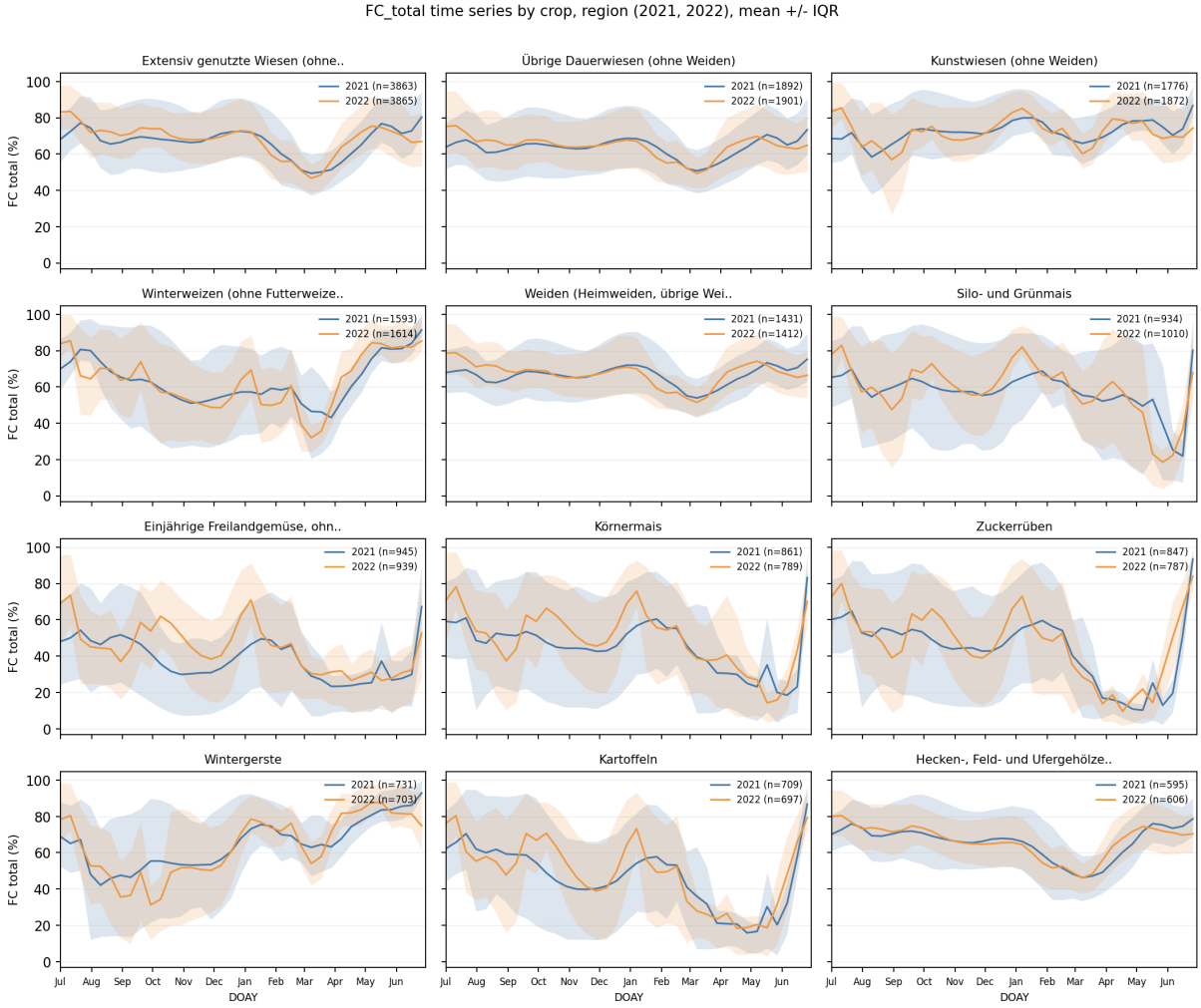


Figure 11: Average timeseries of fractional cover of photosynthetic vegetation (PV), non-photosynthetic vegetation (NPV) and their sum (FC total) in agricultural years 2021 and 2022 (DOAY: day of agricultural year) in Seeland, for the most common crop types.

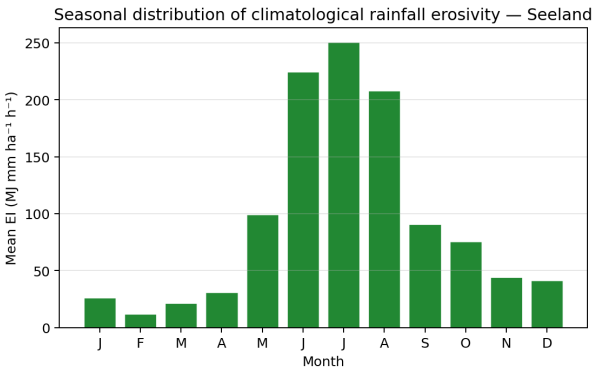


Figure 12: Monthly average erosivity index (based on climatology 2004-2024).

are higher for many crop whereas the ML model follows more closely the trends produced by the previous C-factors. The ML results are slightly higher for most cases except vegetables ("Einährige Freilandgemüse"), where the empirical model and previous C-factors match more. Indeed, the results from Figure 3 show that the ML model slightly underestimates C-factors for this class, and the empirical approach performs better (Figure 1). For potatoes ("Kartoffeln") and fodder ("Futterwiezen"), the ML obtains slightly lower results than the previous C-factors. In 2022, the C-factors obtained with either model are higher than the reference C-factors for all crops, except for vegetables ("Einährige Freilandgemüse") where again the ML value is slightly lower. The higher C-factors are aligned with expectations from Section 12.1. In general, the ML obtained values are closer to the reference and there is a risk of overestimation by the empirical model.

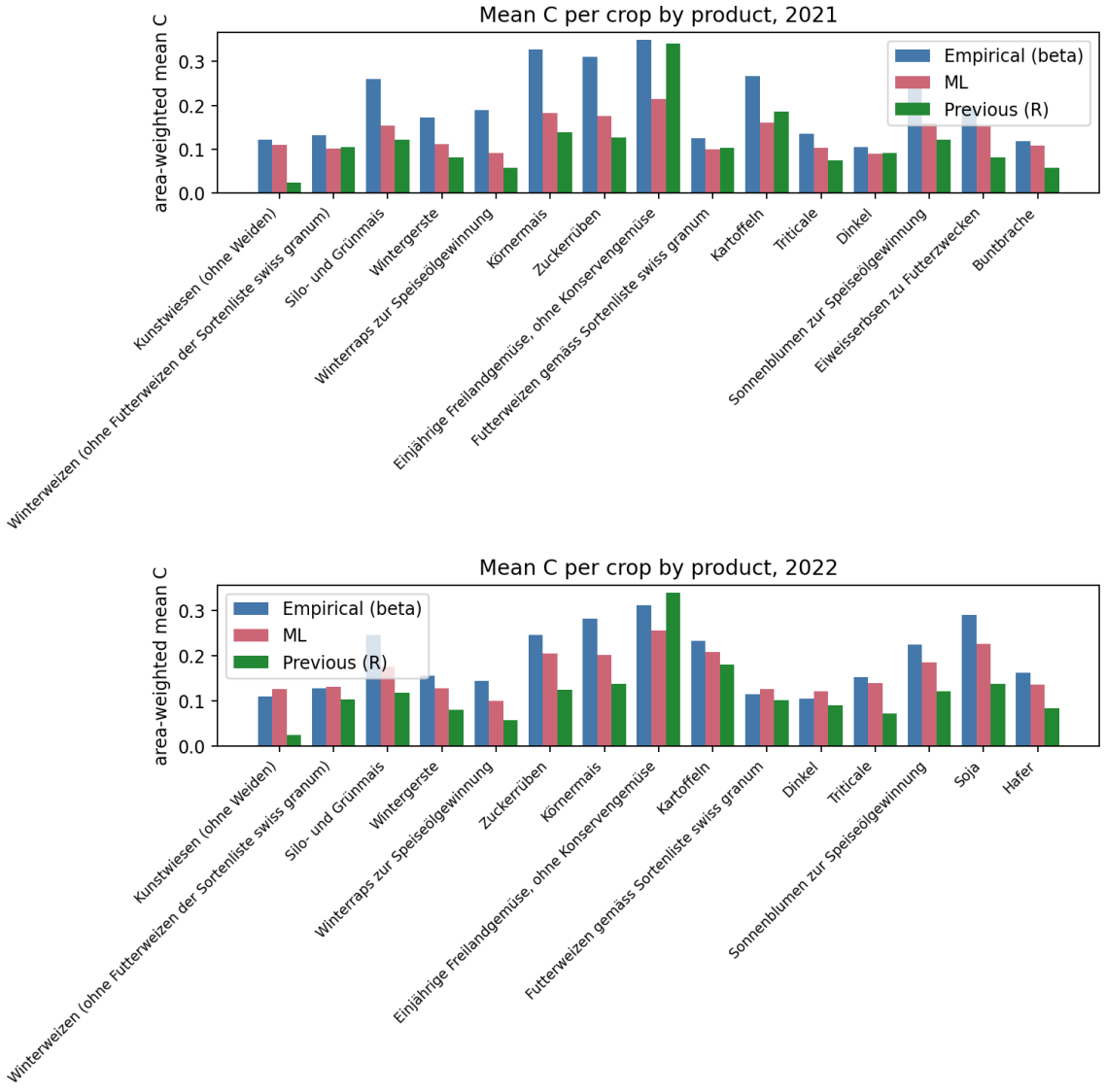
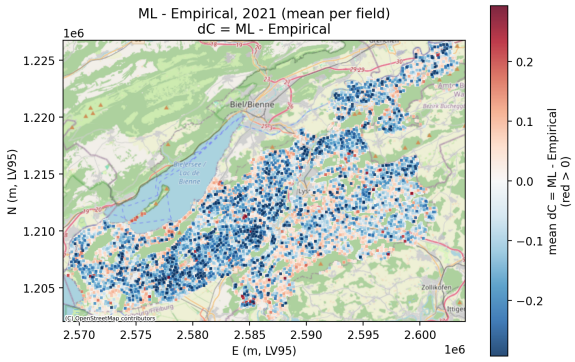


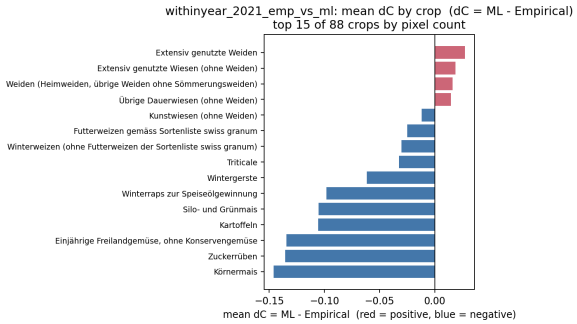
Figure 13: Mean C-factor per crop predicted with the ML model, empirical model and the previous methodology in years 2021 (top) and 2022 (bottom). Only the top 15 crops according to area that year are shown, since the preivous C-factors are not at pixel-level.

We can observe the spatial trends in Figures 14a and 15a where the differences between the ML and empirical approach are plotted. In 2021, the empirically obtained C-factors are higher for most fields (Figure 14a). The differences reported in Figure 14b are largest for maize, sugar beet, vegetables and potatoes. Instead, in 2022, the spatial split becomes more important with

some areas marked by higher ML predictions, while others remain lower (Figure 15a). The ML still predicted lower C-factors for maize and vegetables, but instead predicts higher values for grasslands.

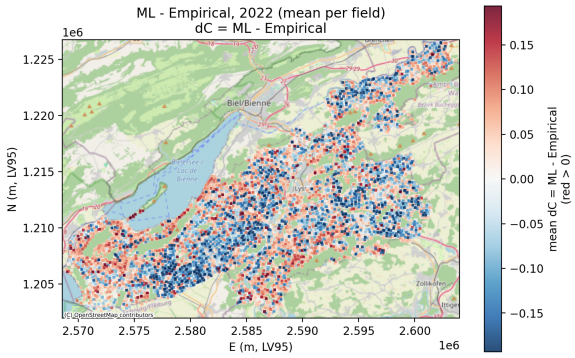


(a) Field mean difference, with only field centroid plotted.

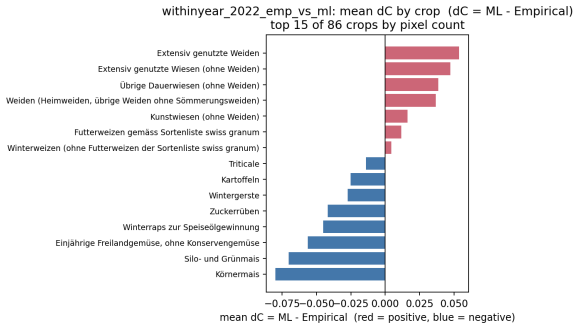


(b) Per crop difference.

Figure 14: Difference in C-factors (ML versus empirical method) in 2021.



(a) Field mean difference, with only field centroid plotted.



(b) Per crop difference.

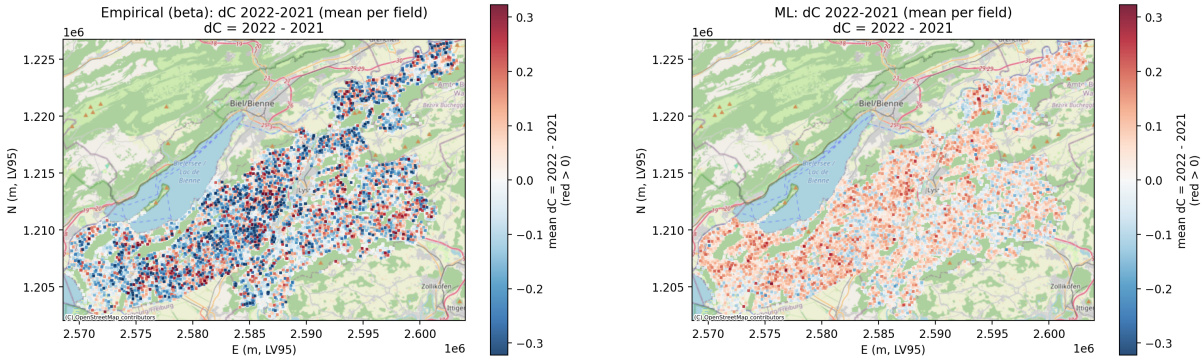
Figure 15: Difference in C-factors (ML versus empirical method) in 2022.

The C-factors obtained generally correspond to what we expect when looking at the fractional cover timeseries (Figure 11). For example, grasslands have high FC over the year and low C-factors, whereas vegetables have some of the highest C-factors and stay around only 50% soil coverage. Other crops like potatoes, maize and sugar beets also have a quite bare spring which leads to higher C-factors.

12.3 Year to year variability

The change in C-factor from 2021 to 2022 is analysed for each product individually and shown in Figure 16. With crop rotations, the same field is unlikely to have the same crop type the following year, leading to changes in C-factor at the field level. The direction of the change is different according to the two developed methods: the ML approach predicts higher C-factors in 2022, whereas with the empirical model a decrease is observed. This indicates fundamentally different responses to changes in fractional cover timeseries. We know from Figure 1 that systematic over- or underestimation is possible with the empirical model, depending on the crop type which could also drive this divergence in response across the two years. The months that have a stronger erosivity index and therefore more crucial for the C-factor are June, July and August. For several crops in Figure 11, the FC dips in August for the year 2022 and is lower also in June 2022, which

would lead to a higher C-factor. Grassland classes have higher C-factors with the ML approach in the warmer 2022 year, although the FC does not change too much. In this case, the ratio between PV and NPV could be driving underlying changes. Overall, the different responses between the empirical model and machine learning model likely stem from limitations in the calibration of the former.



(a) Empirical model

(b) Machine learning model

Figure 16: Difference in field mean C-factor between 2022 and 2021. One point per field centroid is plotted.

As the overall distribution of crops remains stable (see Figure 7), we can still look within a single crop, how the C-factor changed on average. Figure 17 shows how on average the C-factor changed for a crop type. The changes in C-factors obtained with the previous methodology are very small, or slightly negative. Small changes are expected since the C-factors were assigned based only on crop type, tillage method and altitude. For the empirical method, the biggest change occurred for sugar beets, winter rapeseed, and maize. With the ML approach, the changes were less big in amplitude but more present across different crop types. Potatoes, triticale, winter wheat, spelt and vegetables were among the highest.

The change in C-factors also appears to be independent of the change is soil preparation method, as illustrated in Figure 18. Regardless of how the soil preparation changed, the pixels were attributed higher C-factors in 2022 than 2021 for the ML approach and vice-versa with the empirical model.

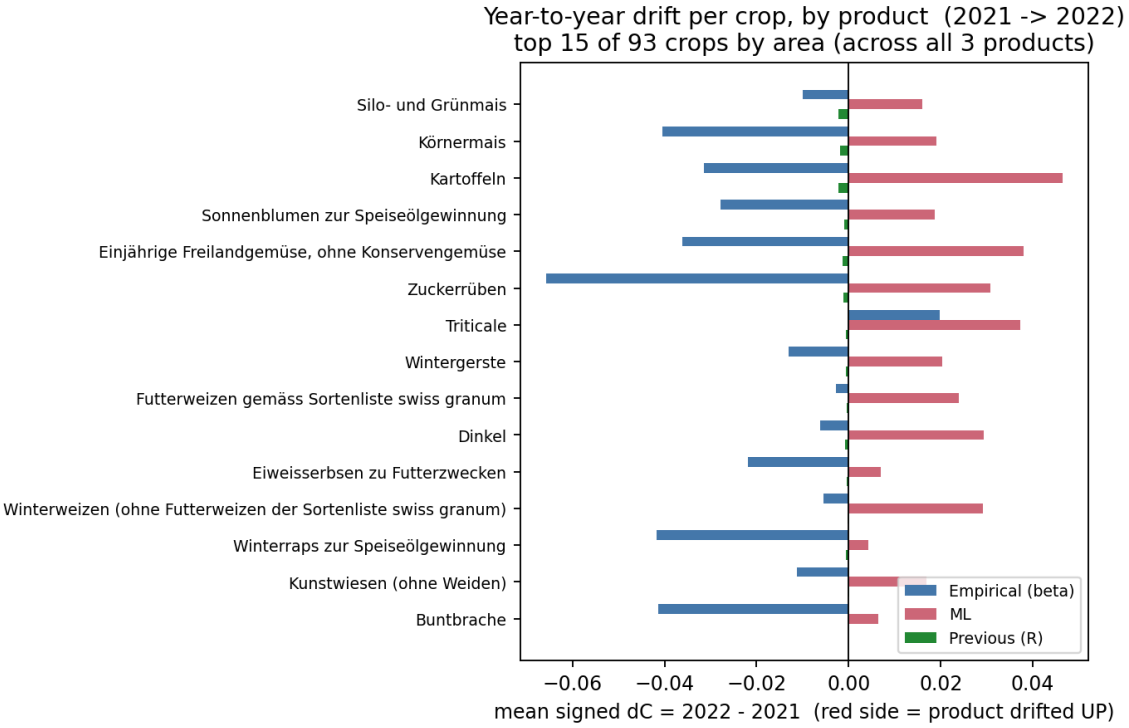


Figure 17: Average change in C-factor between 2021 and 2022 per crop, for the 3 different methods (empirical, ML, tabulated). Only the 15 most prominent crops in terms of area are shown.

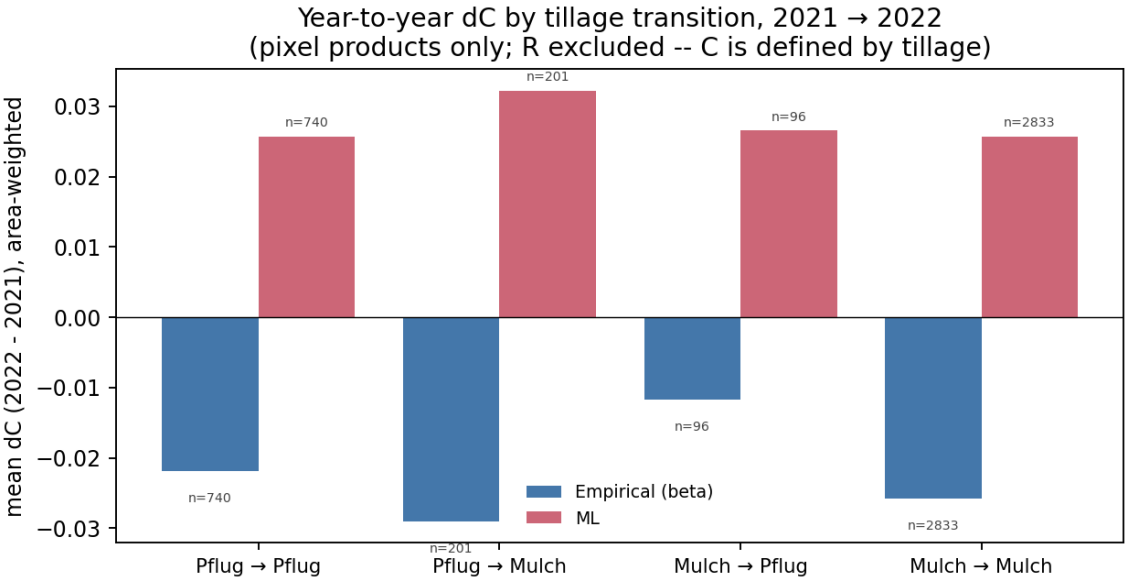


Figure 18: Change in C-factor depending on method (empirical, ML) and soil preparation change.

12.4 Effect of soil groups

The fractional cover predictions were produced using a model specific to different soil types (from a spectral perspective, 5 soil groups across Switzerland, see details in the Fractional Cover report and the documentation. The classification of the soils in the Seeland district are shown in Figure 19.

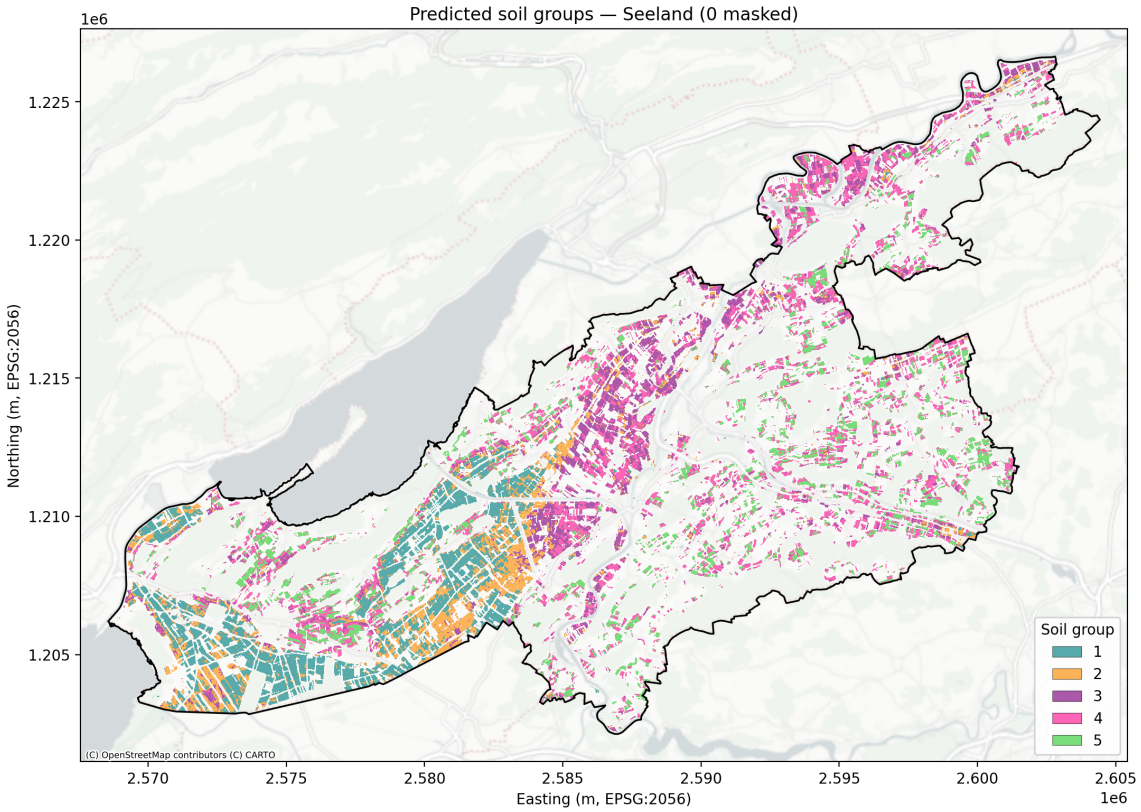


Figure 19

To see whether the soil groups influence the fractional cover predictions and consequently the C-factor, we plotted mean C-factors obtained in the district using the 3 approaches and separated them by soil group. This is shown in Figure 20. There appears to be no strong trend or soil group driving differences in the C-factors, and we therefore can rather focus on looking into other drivers such as crop types, weather and management practices to explain the C-factors.

12.5 Actual erosion risk

To look at where the products disagree on actual erosion risk, the C-factor must be combined with the potential erosion risk and the P-factor. The analysis described here compares the resulting risk at the field level. Figure 21 shows the potential erosion risk map over the area of interest.

The potential erosion risk is taken from the Erosionsrisikokarte des Ackerlandes (`/mnt/Data-Labo-RE/27_Natural_Resources-RE/321.4_WAUM_protected/Daten/ersk_acker22fl/erskacker22fl1.tif`), which is at a 2 m resolution and resampled to match the same 10 m grid of the satellite-derived products. For each field polygon, a single per-polygon potential risk is computed as the mean of the 10 m values falling within the polygon. This value is shared across all three products as a common baseline, so that any cross-product difference in actual risk is attributable to the C-factor alone. For the empirical and ML products, the polygon's actual risk is the area-mean of $(C * pot\_risk)$ over the polygon's 10 m pixels, multiplied by P. For the previous product, where C is by construction constant inside a polygon (it is a lookup keyed by farm × crop × tillage × altitude), the per-polygon actual risk is simply $pot\_risk\_poly * C\_R * P$. C_R is the area-weighted mean of C reported per farm and per crop. The P-factor is set to 0.8, matching the value currently used.

Figure 22 shows the actual erosion for the 3 different products across 2 years. First of all, the

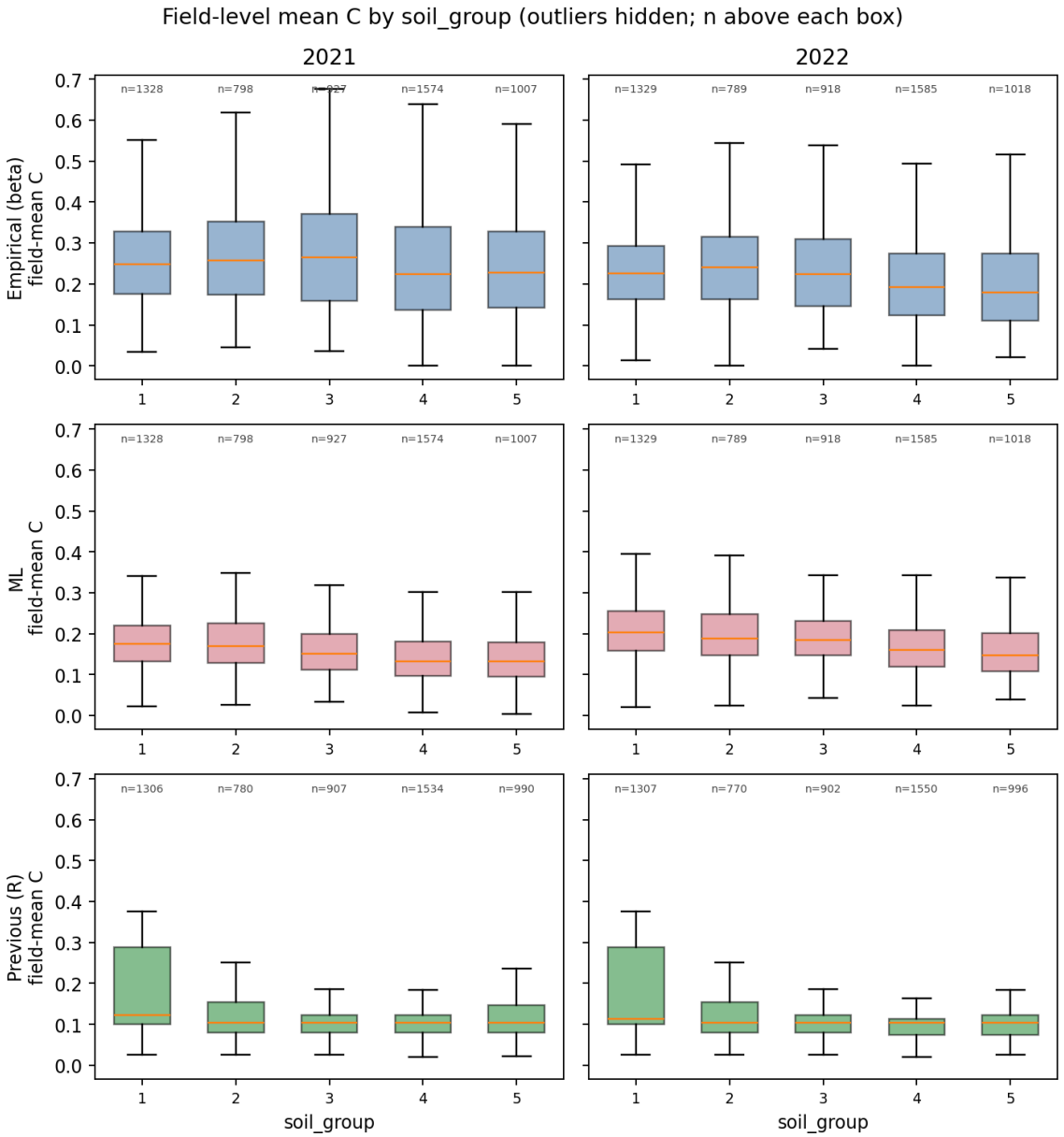


Figure 20: Boxplot of field-mean C-factors produced by empirical model, ML model and previous methodology in 2021 and 2022, split according the soil group. The soil group was determined during fractional cover prediction, to use tailored models for 5 spectrally different soil types in Switzerland (see documentation on Fractional Cover).

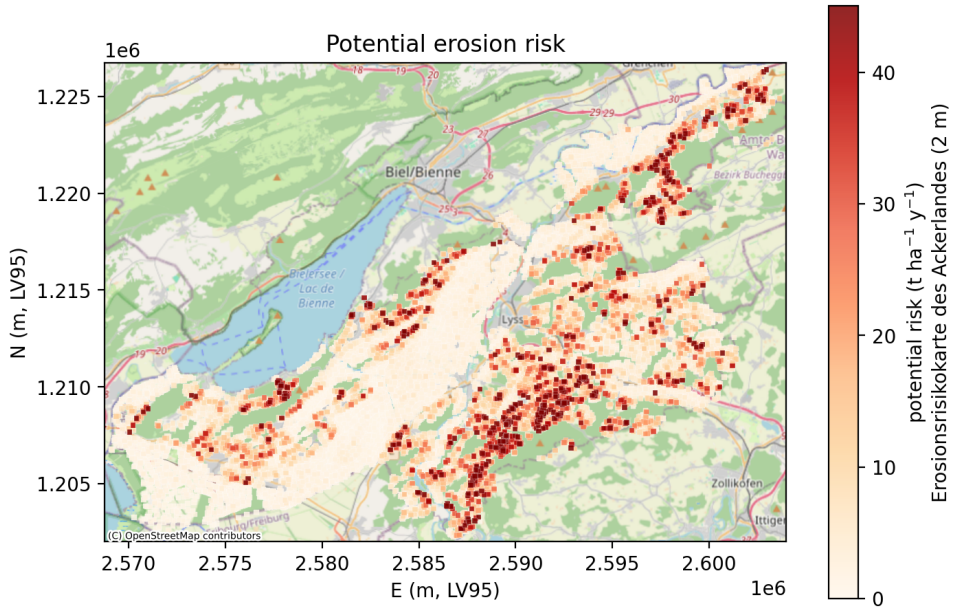


Figure 21: Potential erosion risk in Seeland.

spatial pattern of actual erosion risk changes little from the potential risk regardless of product or year, indicating that overall the main driver are the underlying topographic and environmental conditions. The multiplication by the C-factor leads to actual erosion risk between 0 and 4 t ha^{-1} y^{-1}. From one year to another there are localised changes although the pattern stays stable over the region as a whole. Most notably, the actual erosion risk obtained with the previous methodology is lower than what is obtained using the two satellite-derived products. This is the main difference: updating the method with fractional cover data generally leads to higher C-factors and consequently higher actual erosion risk.

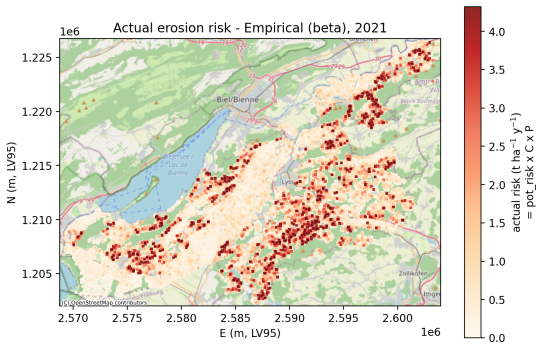
The crops leading to the biggest difference between the empirical/ML model and the previous product are shown in Figure 23 and 24. On average the ML approach lead to an increase of around 0.2 t ha^{-1} y^{-1} in 2022 with respect to the previous results, compared to 0.3 t ha^{-1} y^{-1} for the empirical approach. There are outliers where the increase in actual erosion risk is very high such as for millet ("Hirse") and spice/medicinal plants ("Einjährige Gewürz und Medizinalpflanzen"). This can be expected since such crops represent a minority of the arable area in Switzerland and were not included in the development of the C-factor models. For other more typical crops (maize, sugar beet, barley), the increase ranges from 0.2 to 0.6 t ha^{-1} y^{-1}. In 2021, the increase in actual erosion risk is considerably higher for many crops whereas the ML approach yields smaller changes. This is consistent with the high C-factors predicted by the empirical model.

For each product (empirical, ML, previous), the per-polygon actual erosion risk AR was decomposed using the identity $\mathrm{AR} = \overline{\mathrm{pot}} \cdot \mathrm{amp}$, where $\overline{\mathrm{pot}}$ is the mean potential risk and amp the $C \times P$ amplification. On the log scale this becomes additive:

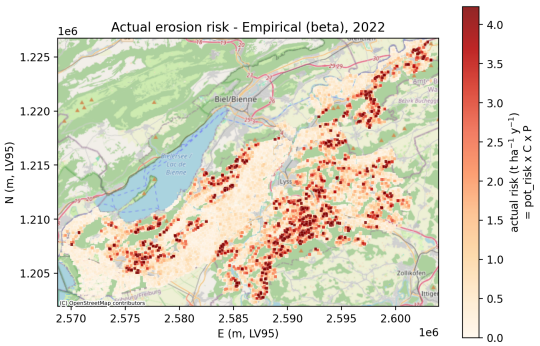
$$\mathrm{Var}(\log \mathrm{AR}) = \mathrm{Var}(\log \overline{\mathrm{pot}}) + \mathrm{Var}(\log \mathrm{amp}) + 2\ \mathrm{Cov}(\log \overline{\mathrm{pot}},\ \log \mathrm{amp}),$$

so the spread in actual risk between fields can be split into shares from *topography* ($\overline{\mathrm{pot}}$), *management* (amp), and their *covariation*. Because $\overline{\mathrm{pot}}$ is constant, between-product differences live entirely in the management and covariation shares.

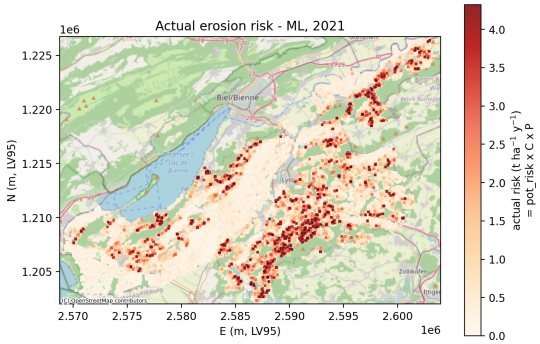
Figures 25 and 26 show the results of this decomposition for years 2021 and 2022 respectively. The left panel shows the three variance shares per product as grouped bars, summing to 1 by construction (covariation can be negative). The right panel shows, for each product, the



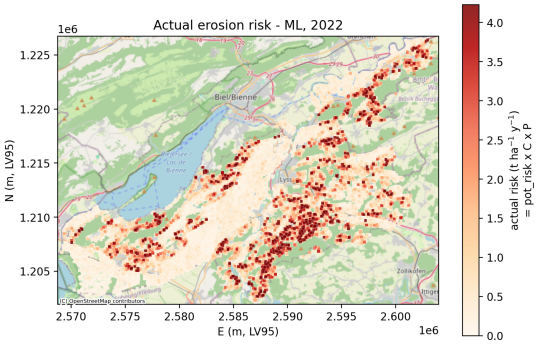
(a) 2021 empirical



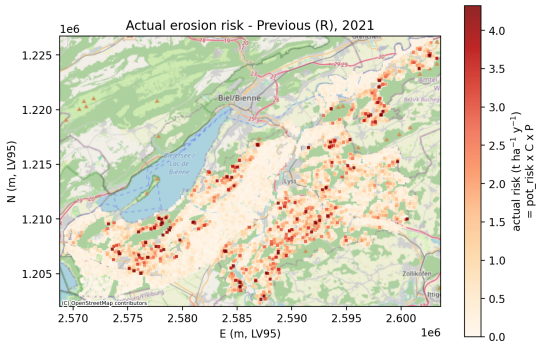
(b) 2022 empirical



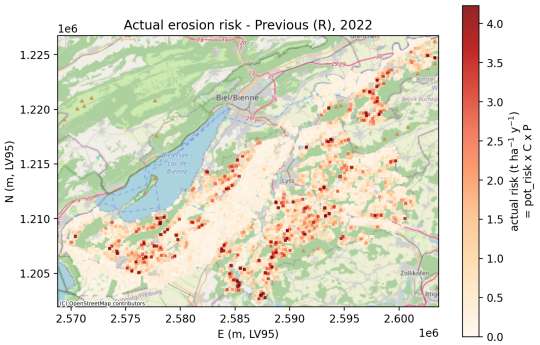
(c) 2021 ML



(d) 2022 ML



(e) 2021 previous



(f) 2022 previous

Figure 22: Actual erosion risk [$\mathrm{t\,ha^{-1}\,y^{-1}}$] in 2021 and 2022, using C-factors from the empirical model, ML model and previous results. On value is shown per field at it's centroid.

univariate linear R^2 of AR $\sim$ pot_risk and AR $\sim C \times P$, i.e. how well each driver alone predicts actual risk. The results show that the potential erosion risk is the main driver of AR, whereas the management part plays a smaller explanative role. Among the 3 products, the contribution of management is lowest for the ML product.

Under all three C-factor products, between-field variation in actual erosion risk is dominated by topography: potential risk alone explains 52% (empirical), 48% (previous), and 83% (ML) of the variance in log(AR) across fields. The management term contributes a secondary share, with roughly 40–48% for the empirical and previous products, but only 16% for ML. This is supported by low R^2 values between AR and $C \times P$, indicatin that the C-factor alone is a weak predictor of actual risk. This also means that the ML C-factor varies much less between fields than the empirical one. Future work could investigate whether the results are still in the expected range,

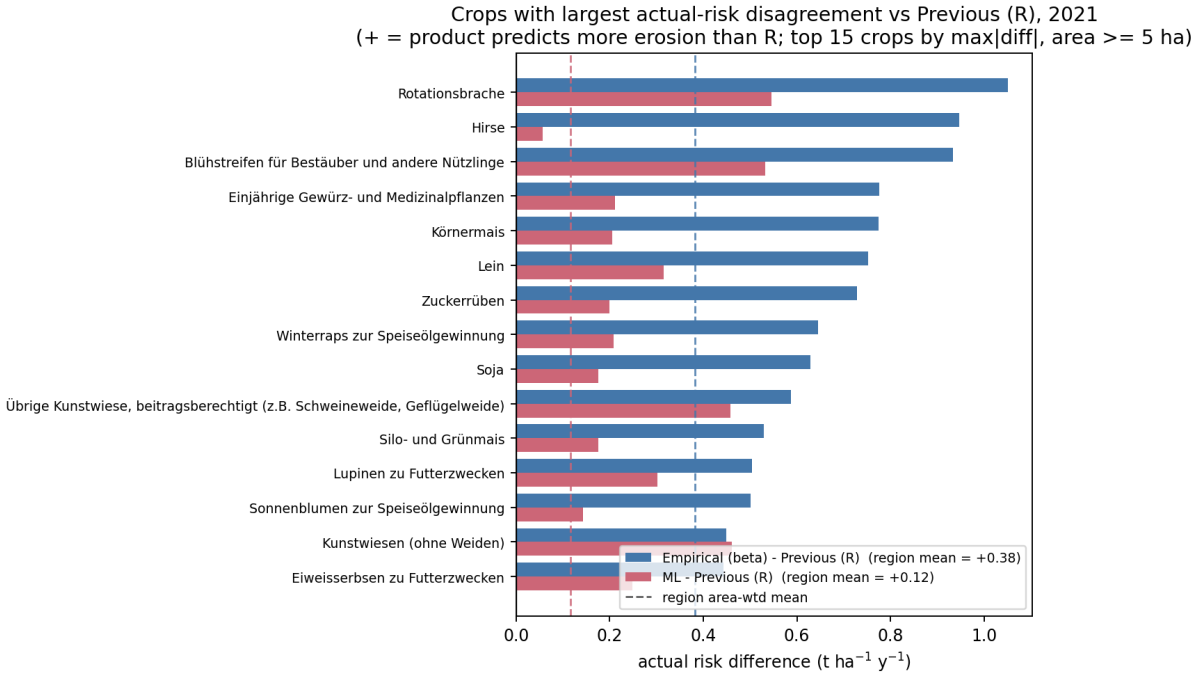


Figure 23: Top 15 crops leading to biggest differences in actual erosion risk with respect to previous approach in 2022.

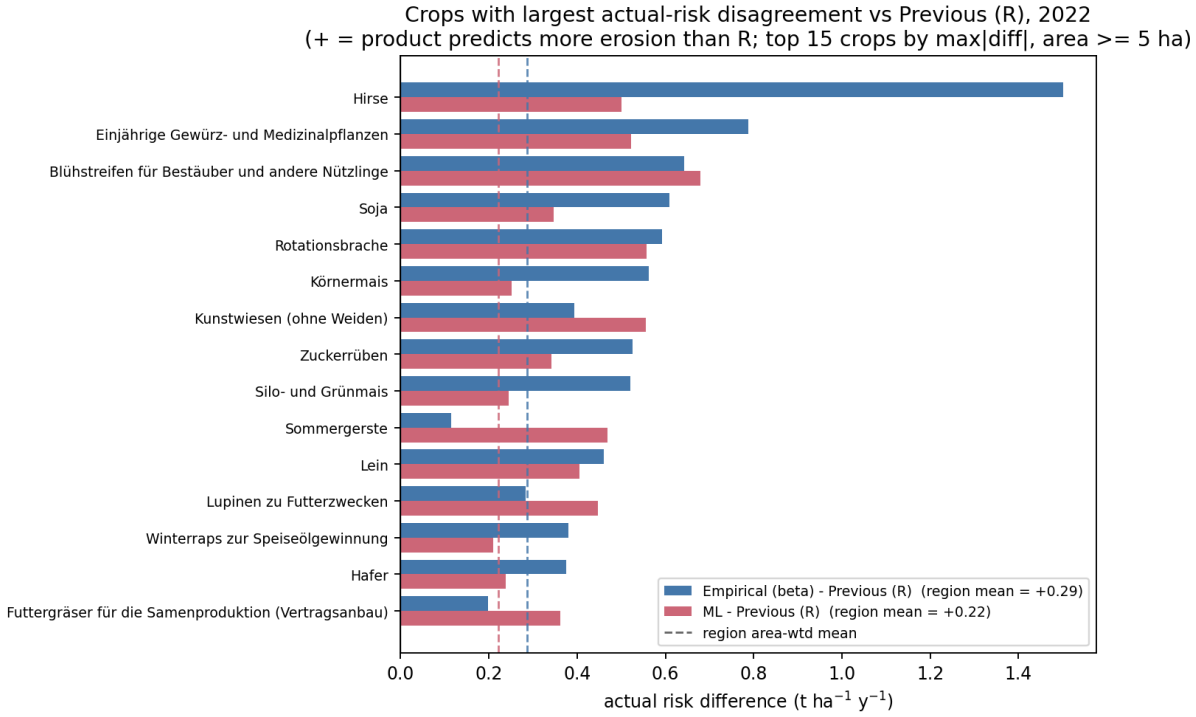


Figure 24: Top 15 crops leading to biggest differences in actual erosion risk with respect to previous approach in 2022.

and ensure that the ML model does not pull towards the mean, in which case retraining should ideally be done with more samples per stratum to obtain more variability.

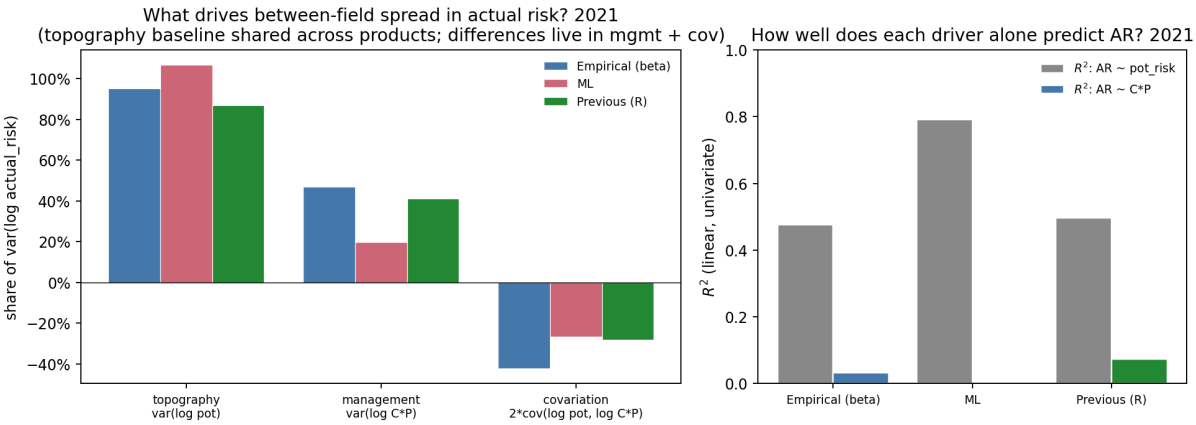


Figure 25: Variance decomposition of actual erosion risk in 2021.

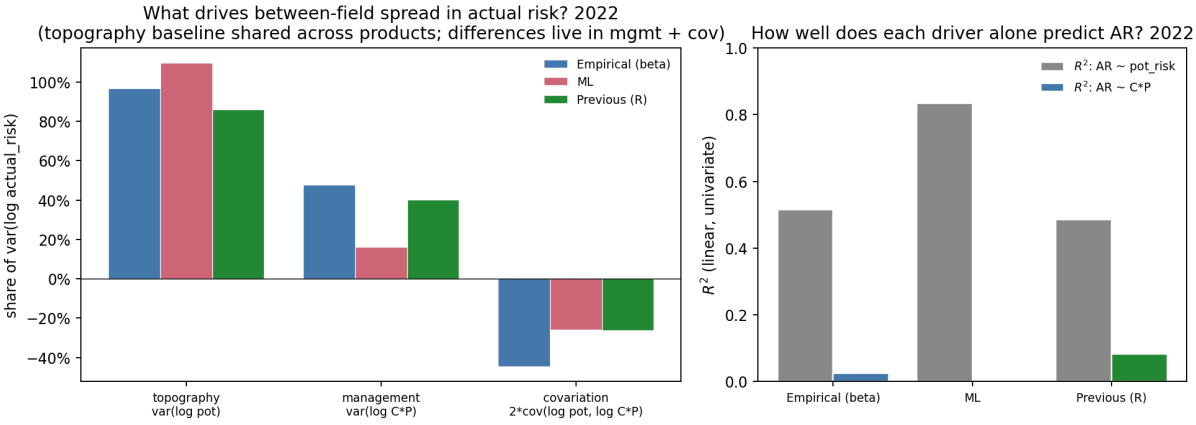


Figure 26: Variance decomposition of actual erosion risk in 2022.

12.6 Reproducing the Analysis

All scripts are located in the `cfactor_analysis/` directory and should be executed from that directory. The five Python scripts are designed to be run sequentially; each writes its outputs to a dedicated subdirectory so results do not overwrite one another. Scripts 3–5 read the per-pixel parquet files produced by the upstream `compute_cfactor_pixels.py` and `compute_cfactor_pixels_ml.py` pipelines and do not recompute the C-factor themselves; those upstream jobs must be completed first.

`plot_soilgroups_seeland.py` mosaics the DLR soil-suite predictions (`SRC_*.zarr` tiles) over the Seeland region, masks invalid pixels, and overlays the result on a Swisstopo national colour-map basemap. Output is written to `seeland_soilgroups.png`.

```
python plot_soilgroups_seeland.py
```

`analyse_area.py` characterises the study area for 2021 and 2022: MeteoSwiss climate series (rainfall, temperature, sunshine), crop and tillage distributions from the LNF geodatabase, the reported C-factor landscape, rainfall erosivity (EI_{30}), Sentinel-2 data quality, and potential and P-combined erosion-risk maps. Figures are written to `figures/`. Two optional flags skip slow data-loading steps:

```
python analyse_area.py
python analyse_area.py --skip-meteo    # omit MeteoSwiss loading
python analyse_area.py --skip-s2       # omit pixel-quality section
```

`compare_products.py` compares the three C-factor products across 2021 and 2022. Analyses cover inter-year stability, cross-product disagreement stratified by crop, elevation, municipality and tillage practice, and the FC time-series context. Figures and summary CSVs are written to `figures_compare/`.

```
python compare_products.py
```

`cfactor/extract_fc_pixels.py` In order to plot and analyse the FC timeseries in **`compare_products.py`**, the FC timeseries of pixels in the Seeland region need to be saved. This script needs to be run to save the FC timeseries before the analysis. The results are inparquet files to `cfactor/output/fc_pixels/`

```
python extract_fc_pixels.py #in cfactor/ directory
```

`compute_actual_risk.py` computes actual erosion risk ($R \times K \times LS \times C \times P$) at field level for all three C-factor products, using the *Erosionsrisikokarte des Ackerlandes* as the potential-risk source and $P = 0.8$. It produces per-product risk maps, signed pairwise difference maps, hotspot tables, and per-crop breakdowns. Outputs are written to `figures_actualrisk/`.

```
python compute_actual_risk.py
```

`analyse_soil_effect.py` investigates whether soil group influences the C-factor by joining per-pixel parquets with a nearest-neighbour soil-group lookup and aggregating to field level. It produces boxplots, per-crop heatmaps, and Kruskal–Wallis tests for each product–year combination. Outputs are written to `figures_soil/`. Optional flags restrict the years processed or skip the R baseline (previous C-factors):

```
python analyse_soil_effect.py
python analyse_soil_effect.py --years 2021 2022
python analyse_soil_effect.py --skip-r
```

A Glossary

AGIS Agrarstatistisches Informationssystem – Swiss agricultural statistics information system.

ALR Additive Log-Ratio: a compositional data transform.

Berg High-altitude stratum (> 600 m a.s.l.) in the Swiss C-factor table.

EI Erosivity Index; also rainfall erosivity factor R in RUSLE.

FC Fractional Cover: proportions of photosynthetically active vegetation (PV), non-photosynthetic vegetation (NPV), and bare soil.

GPR Gaussian Process Regression.

LNF Landwirtschaftliche Nutzfläche – agricultural land-use polygon maps issued annually by the Swiss cantons.

LOYO Leave-One-Year-Out cross-validation.

MLP Multilayer Perceptron.

NPV Non-photosynthetic vegetation (residues, senescent material).

PV Photosynthetically active (green) vegetation.

REB Ressourceneffizienzbeitrag – resource efficiency payment table recording tillage method per farm-year.

RUSLE Revised Universal Soil Loss Equation.

SLR Soil Loss Ratio: $\mathrm{SLR}(t) = \exp(-\beta \cdot \mathrm{FC}(t))$.

Tal Low-altitude stratum (≤ 600 m a.s.l.) in the Swiss C-factor table.

References

Matthews, F., Verstraeten, G., Borrelli, P., and Panagos, P. (2023). A field parcel-oriented approach to evaluate the crop cover-management factor and time-distributed erosion risk in europe. *International Soil and Water Conservation Research*, 11(1):43–59.

Prasuhn, V., Hutchings, C., Gilgen, A., and Blaser, S. (2023). Der Agrarumweltindikator Erosionsrisiko, kulturspezifische C-Faktoren sowie eine Karte des aktuellen Erosionsrisikos der Schweiz. Technical report, Agroscope.